LAPORAN PRAKTIKUM

Modul 9: HMVC, Otorisasi, dan AJAX pada React Starter Kit



Disusun Oleh:

Nama : Davin Aileenskie Sofiandy

NIM : 707012500135

Kelas : D4SIKC-49-04

Mata Kuliah : Pengembangan Aplikasi Berbasis Web

TELKOM UNIVERSITY

FAKULTAS ILMU TERAPAN

JURUSAN S1 TERAPAN SISTEM INFORMASI KOTA CERDAS

TAHUN AJARAN 2026

1. Tujuan Praktikum

Praktikum Modul 9 ini saya kerjakan pada project PRK_TA_9, yaitu aplikasi perpustakaan SiPustaka yang dibangun dengan Laravel React Starter Kit. Aplikasi ini sudah saya pakai sebagai tugas akhir praktikum, jadi isinya bukan satu modul contoh, melainkan tiga modul yang saling melengkapi: Buku, Peminjaman, dan UserManagement. Di atas ketiganya ada autentikasi bawaan starter kit, dashboard, dan layout React yang tetap berada di luar folder Modules.

Fokus Modul 9 ada tiga: memecah kode ke dalam HMVC, membatasi akses berdasarkan role, dan membuat CRUD AJAX dengan unggah berkas. Saya menerapkan ketiganya pada fitur yang benar-benar dipakai, bukan pada tabel uji coba. Modul Buku dan UserManagement memakai CRUD Inertia, sedangkan Peminjaman memakai fetch() dan respons JSON. Dengan begitu, saya bisa membandingkan dua pendekatan CRUD dalam satu aplikasi.

Tujuan yang ingin saya capai:

1. Memasang nwidart/laravel-modules pada Laravel React Starter Kit dan memecah fitur SiPustaka menjadi modul Buku, Peminjaman, dan UserManagement.
2. Menambahkan kolom role pada tabel users, membuat enum UserRole, dan memasang middleware EnsureUserHasRole pada route yang perlu dibatasi.
3. Memakai autentikasi Fortify bawaan starter kit untuk register, login, verifikasi email, dan logout, lalu memastikan akun baru selalu berrole user.
4. Membuat CRUD Buku dengan Inertia, pencarian, pagination, dan pembatasan tombol aksi hanya untuk admin.
5. Membuat CRUD UserManagement yang hanya bisa dibuka admin, lengkap dengan pencarian, filter role, dan pengaman agar admin tidak menurunkan role akunnya sendiri.
6. Membuat CRUD Peminjaman berbasis AJAX dengan fetch(), respons JSON, unggah bukti lewat FormData, serta penggantian dan penghapusan berkas di storage.
7. Menampilkan pesan berhasil dari flash session dan pesan error per input, baik dari validasi Inertia maupun dari respons 422 JSON.
8. Menguji role, middleware, CRUD ketiga modul, unggah berkas, dan seeder dengan Pest sampai seluruh tes lulus.

2. Alat dan Bahan

Praktikum ini saya kerjakan di laptop pribadi ASUS ROG Strix G614JU dengan Windows 11 Pro 10.0.26200, prosesor Intel Core i7-13650HX, dan RAM 16 GB. Project berada di D:\xampp\htdocs\PABW\Praktikum_9\PRK_TA_9. Saya memakai XAMPP untuk Apache dan MySQL, lalu menjalankan Laravel dengan php artisan serve di http://127.0.0.1:8000. Database yang dipakai bernama pertemuan9.

2.1 Perangkat Lunak

Versi di bawah ini saya catat dari mesin yang saya pakai, bukan dari dokumentasi. Bagian frontend saya tulis terpisah karena paketnya cukup banyak dan tidak semuanya dipakai langsung di halaman modul.

Tabel 1. Versi perangkat lunak yang dipakai selama praktikum.

Perangkat Lunak	Versi / Keterangan
Windows 11 Pro	10.0.26200, 64-bit
XAMPP (Apache & MySQL)	Terpasang di D:\xampp
MySQL / MariaDB	10.4.32-MariaDB, database pertemuan9
phpMyAdmin	Pemeriksaan tabel users, bukus, dan peminjamans
PHP	8.2.12 (ZTS Visual C++ 2019 x64)
Composer	2.10.3
Laravel Framework	12.71.0
nwidart/laravel-modules	12.0.5
Inertia, Fortify, Ziggy	inertia-laravel 2.0.28, Fortify 1.41.0, Ziggy 2.6.5
React dan TypeScript	React 19.0.0, @inertiajs/react 2.0.3, TypeScript 5.7.3
Tailwind CSS dan Vite	Tailwind CSS 4.0.8, Vite 6.1.1
Pest	3.8.7
Visual Studio Code	Editor kode dan terminal project
Git	2.53.0.windows.1
Google Chrome	157.0.8103.52, untuk aplikasi dan DevTools

Untuk tampilan, starter kit memakai React 19 dengan TypeScript, Vite, Tailwind CSS, dan komponen shadcn/ui. Ikon diambil dari lucide-react, sedangkan helper route memakai Ziggy. Saya tidak menambah library UI baru karena komponen yang sudah ada cukup untuk tabel, dialog, badge, dan form.

2.2 Bahan

Bahan utamanya adalah Laravel React Starter Kit yang sudah terpasang beserta autentikasinya. Route login, register, logout, verifikasi email, dan reset password berada di routes/auth.php dan ditangani Fortify, bukan di dalam modul. Saya juga memakai paket nwidart/laravel-modules untuk membuat folder modul, Pest untuk pengujian, dan satu berkas contoh bukti peminjaman berformat PNG. Untuk uji validasi berkas saya menyiapkan satu berkas ZIP kecil yang sengaja ditolak server.

Data awal yang dipakai untuk demonstrasi adalah 6 buku, 4 peminjaman, dan 2 akun. Akun admin@example.com dan user@example.com sama-sama memakai password password. Data itu dibuat lewat seeder supaya tampilan dashboard dan tabel langsung berisi.

3. Langkah-Langkah Praktikum

Urutan pengerjaan saya mulai dari menyiapkan project dan modul, lalu role, lalu ketiga modul, dan terakhir pengujian. Saya sengaja mengerjakan Buku lebih dulu karena polanya paling sederhana, lalu UserManagement karena ada aturan tambahan, dan Peminjaman karena bagian AJAX dan unggah berkas paling banyak makan waktu.

3.1 Menyiapkan Project dan Modul HMVC

Project ini sudah saya pakai sejak modul sebelumnya, jadi yang saya lakukan adalah memastikan struktur dan databasenya siap. Nama folder project adalah PRK_TA_9 dan database yang aktif bernama pertemuan9. Materi modul menyebut nama project sipustaka_hmvc_react dan database db_sipustaka; saya tidak mengganti nama folder dan database yang sudah berjalan karena semua data dan konfigurasi sebelumnya ada di sana. Perbedaan nama ini saya catat supaya tidak dianggap terlewat.

Paket modul dipasang dengan Composer, lalu tiga modul dibuat lewat Artisan. Setelah itu saya memeriksa statusnya dengan php artisan module:list. Ketiga modul harus berstatus Enabled sebelum route-nya bisa dipanggil.

```
php artisan module:list

[Enabled] Buku            Modules/Buku            [0]
[Enabled] Peminjaman      Modules/Peminjaman      [0]
[Enabled] UserManagement  Modules/UserManagement  [0]
```

Output 1. Hasil pemeriksaan status modul.

Path lengkap ketiga modul berada di D:\xampp\htdocs\PABW\Praktikum_9\PRK_TA_9\Modules. Di blok output saya tulis relatif agar tidak terpotong, tetapi nama dan statusnya sama dengan yang keluar di terminal.

Lalu saya menghubungkan halaman React milik modul ke resolver Inertia. Di starter kit, halaman biasa berada di resources/js/pages. Supaya halaman modul juga ikut terdeteksi Vite, saya menambahkan pola ../../Modules/*/resources/js/pages/**/*.tsx ke import.meta.glob. Nama

halaman modul memakai format Buku::index, Peminjaman::index, dan UserManagement::index. Resolver memecah nama itu, lalu menyusun path sesuai folder modul.

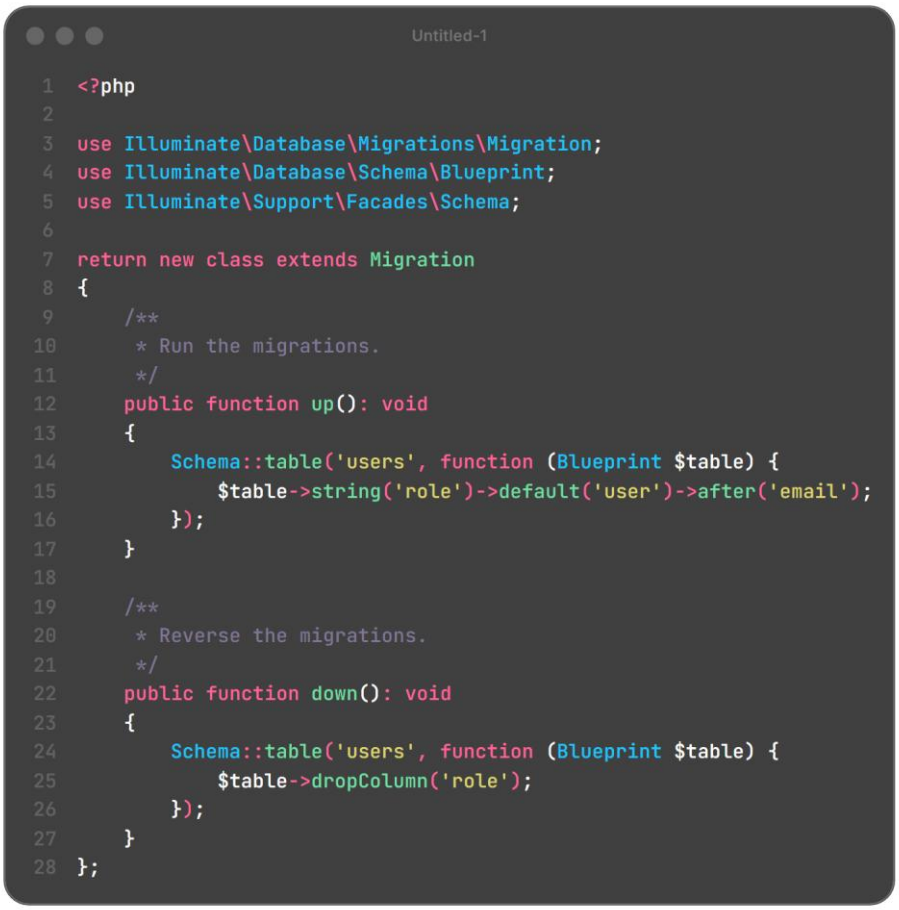


Gambar 1. Resolver halaman modul di resources/js/app.tsx.

Konsekuensinya, controller modul tidak perlu menyebut path file React secara lengkap. Cukup Inertia::render('Buku::index', ...) dan resolver yang mengarahkan ke Modules/Buku/resources/js/pages/index.tsx. Saya memakai pola ini di ketiga modul supaya penamaan halaman konsisten.

3.2 Menambahkan Role dan Enum

Role disimpan sebagai string di kolom role pada tabel users. Saya memilih string, bukan tabel relasi, karena hanya ada dua peran yang dipakai: admin dan user. Nilai default-nya user, jadi akun yang mendaftar lewat register otomatis mendapat akses biasa tanpa perlu diisi manual.



Gambar 2. Migration yang menambahkan kolom role pada tabel users.

Nilainya di PHP saya bungkus dalam enum UserRole. Dengan enum, saya tidak perlu menghafal string admin atau user di controller dan request. Kalau nanti ada typo, kesalahannya lebih cepat terlihat.

Untitled-1

```php
<?php

namespace App\Enums;

enum UserRole: string
{
    case Admin = 'admin';
    case User = 'user';

    /**
     * Get the human readable label of the role.
     */
    public function label(): string
    {
        return match ($this) {
            self::Admin => 'Admin',
            self::User => 'User',
        };
    }
}
```

Gambar 3. Enum UserRole beserta label yang dipakai di tampilan.

Model User memberi cast role ke enum, lalu menyediakan dua helper: isAdmin() dan hasRole(). Helper ini yang dipakai di middleware, request, controller, dan komponen React lewat shared props. Saya juga menambahkan state admin() pada UserFactory supaya pengujian bisa membuat akun admin tanpa mengisi role manual.



Gambar 4. Cast role dan helper isAdmin() serta hasRole() pada model User.

3.3 Middleware EnsureUserHasRole

Pembatasan akses saya taruh di satu middleware. Middleware ini membaca route parameter role, memecahnya dengan koma, lalu memeriksa apakah user yang login memiliki salah satu role tersebut. Kalau tidak, request dihentikan dengan 403. Saya memilih satu middleware dengan parameter, bukan tiga middleware terpisah, supaya route tinggal menulis role:admin atau role:admin,user.

```php
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class EnsureUserHasRole
{
    /**
     * Ensure the authenticated user has one of the given roles.
     *
     * Usage: ->middleware('role:admin') or ->middleware('role:admin,user')
     *
     * @param  Closure(Request): (Response)  $next
     */
    public function handle(Request $request, Closure $next, string ...$roles): Response
    {
        $user = $request->user();
        if ($user === null || ! $user->hasRole(...$roles)) {
            abort(403, 'Anda tidak memiliki hak akses ke halaman ini.');
        }

        return $next($request);
    }
}
```

Gambar 5. Middleware EnsureUserHasRole memeriksa role user yang sedang login.

Middleware didaftarkan dengan alias role di bootstrap/app.php. Setelah itu route group bisa memakai alias tersebut. Saya juga memakai middleware auth lebih dulu, jadi user yang belum login diarahkan ke halaman login, sedangkan user yang sudah login tetapi role-nya tidak cocok mendapat 403.

```php
    ->withMiddleware(function (Middleware $middleware): void {
        $middleware->encryptCookies(except: ['appearance', 'sidebar_state']);

        $middleware->web(append: [
            HandleAppearance::class,
            HandleInertiaRequests::class,
            AddLinkHeadersForPreloadedAssets::class,
        ]);

        $middleware->alias([
            'role' => EnsureUserHasRole::class,
        ]);
    })
```

Gambar 6. Alias middleware role di bootstrap/app.php.

3.4 Modul Buku

Modul Buku menangani katalog buku. Route-nya memakai middleware auth dan role:admin,user untuk halaman index, lalu role:admin untuk create, store, edit, update, dan destroy. Saya menulis batasan itu di route, lalu mengulanginya di StoreBukuRequest dan

UpdateBukuRequest lewat authorize(). Dua lapis ini penting karena tombol di React bisa disembunyikan, tetapi request tetap bisa dikirim langsung.



Gambar 7. Route modul Buku dengan pembatasan role yang berbeda untuk baca dan tulis.

Controller index menerima parameter search, lalu memfilter judul, penulis, dan penerbit. Hasilnya dipagination 10 baris dan diurutkan berdasarkan judul. Saya memakai when() supaya query tetap sederhana saat kata kunci kosong. Data yang dikirim ke React hanya kolom yang dipakai tabel.



Gambar 8. Pencarian dan pagination pada BukuController@index.

Untuk create dan update, controller memakai request class yang memvalidasi judul, penulis, penerbit, tahun terbit, dan stok. Tahun terbit dibatasi 1900 sampai tahun depan, sedangkan stok

tidak boleh negatif. Pesan errornya saya tulis dalam bahasa Indonesia agar cocok dengan tampilan. destroy() cukup menghapus baris; karena Buku tidak menyimpan berkas, tidak ada file yang perlu dibersihkan.



Gambar 9. Bagian store dan destroy pada BukuController.

Di sisi React, halaman index menyimpan kata kunci di state, lalu mengirim ulang GET /buku dengan router.get. Saya memakai preserveState dan replace supaya posisi form dan riwayat browser tetap wajar. Tombol Tambah, Edit, dan Hapus hanya dirender kalau isAdmin bernilai true. User biasa tetap bisa mencari dan membaca tabel, tetapi tidak melihat tombol yang bisa mengubah data.



Gambar 10. Pencarian dan pengambilan data awal di halaman Buku.

3.5 Modul UserManagement

Modul UserManagement adalah halaman admin untuk mengelola akun. Semua route-nya berada di prefix admin/users dan memakai middleware auth serta role:admin. Saya memakai resource route kecuali show karena halaman detail tidak dipakai; data yang perlu diedit sudah ada di tabel.



Gambar 11. Route admin/users dengan middleware role:admin.

Controller index menerima search dan role. Pencarian mencakup nama dan email, sedangkan filter role memakai enum. Hasilnya dipagination 10 baris dan diurutkan berdasarkan nama. Saya mengirim daftar role ke React supaya pilihan filter dan form tidak di-hardcode di dua tempat.



Gambar 12. Pencarian, filter role, dan transformasi data di UserManagementController@index.

Bagian update punya dua aturan tambahan. Pertama, password hanya diubah kalau kolomnya diisi; kalau kosong, field password di-unset sebelum update. Kedua, admin tidak boleh menurunkan role akunnya sendiri. Aturan ini saya taruh di UpdateUserRequest, bukan hanya di tampilan, supaya tetap berlaku kalau request dikirim langsung. destroy() juga menolak penghapusan akun sendiri.



Gambar 13. Pembaruan password dan pengaman akun sendiri di UserManagementController.

Di React, filter dikirim lewat router.get dengan parameter search dan role. Tabel menandai baris akun yang sedang login dengan badge Anda, lalu menyembunyikan tombol Hapus untuk baris itu. Kalau admin tetap mencoba menurunkan role sendiri lewat form edit, server mengembalikan error validasi dan pesannya muncul di bawah select role. Saya menguji dua jalur ini: lewat tampilan dan lewat request langsung.



Gambar 14. Filter pencarian dan role di halaman UserManagement.

3.6 Modul Peminjaman

Modul Peminjaman sengaja saya buat berbeda. Kalau Buku dan UserManagement memakai Inertia, Peminjaman memakai fetch() dan respons JSON. Tujuannya supaya saya bisa menunjukkan CRUD AJAX yang benar-benar tidak memuat ulang halaman, termasuk unggah berkas lewat FormData.

Route-nya memakai middleware auth dan role:admin,user, karena admin dan user biasa sama-sama boleh mencatat peminjaman. Ada beberapa route JSON: GET /peminjaman/data untuk daftar, GET /peminjaman/{id}/edit untuk data form, POST /peminjaman untuk tambah, PUT /peminjaman/{id} untuk ubah, dan DELETE /peminjaman/{id} untuk hapus. Route index tetap memakai Inertia untuk kerangka halaman.

```php
<?php

use Illuminate\Support\Facades\Route;
use Modules\Peminjaman\Http\Controllers\PeminjamanController;

Route::middleware(['auth', 'role:admin,user'])->group(function () {
    Route::get('peminjaman', [PeminjamanController::class, 'index'])->name('peminjaman.index');
    Route::get('peminjaman/data', [PeminjamanController::class, 'getData'])->name('peminjaman.data');
    Route::post('peminjaman', [PeminjamanController::class, 'store'])->name('peminjaman.store');
    Route::get('peminjaman/{peminjaman}/edit', [PeminjamanController::class, 'edit'])->name('peminjaman.edit');
    Route::put('peminjaman/{peminjaman}', [PeminjamanController::class, 'update'])->name('peminjaman.update');
    Route::delete('peminjaman/{peminjaman}', [PeminjamanController::class, 'destroy'])->name('peminjaman.destroy');
});
```

Gambar 15. Route modul Peminjaman untuk halaman Inertia dan operasi JSON.

Controller store menyimpan data peminjaman dan berkas bukti. Berkas disimpan di disk public dengan folder bukti_peminjaman, sedangkan nama asli berkas disimpan di kolom nama_bukti. Saya memisahkan dua kolom itu karena path hasil store sudah berupa nama acak; kalau nama asli tidak disimpan, pengguna tidak tahu berkas mana yang mereka unggah.

```php
    public function index(): Response
    {
        return Inertia::render('Peminjaman::index');
    }

    /**
     * Return all loan records as JSON.
     */
    public function getData(): JsonResponse
    {
        return response()->json(Peminjaman::latest()->get());
    }

    /**
     * Store a newly created loan together with its uploaded proof.
     */
    public function store(PeminjamanRequest $request): JsonResponse
    {
        $data = $request->safe()->except('bukti');

        $file = $request->file('bukti');
        $data['bukti'] = $file->store('bukti_peminjaman', 'public');
        $data['nama_bukti'] = $file->getClientOriginalName();

        Peminjaman::create($data);

        return response()->json(['success' => 'Data peminjaman berhasil disimpan.']);
    }
```

Gambar 16. Penyimpanan data dan berkas bukti di PeminjamanController@store.

Update hanya mengganti berkas kalau ada file baru. Kalau tidak, path lama dibiarkan. Saat berkas diganti atau data dihapus, file lama di storage juga dihapus supaya tidak menumpuk.

destroy() menghapus file lebih dulu, baru baris database. Respons sukses dan error dikirim sebagai JSON dengan status yang sesuai.



Gambar 17. Penggantian berkas dan penghapusan data di PeminjamanController.

Validasi Peminjaman saya taruh di PeminjamanRequest. Nama peminjam, judul buku, tanggal pinjam, tanggal kembali, dan status wajib diisi. Tanggal kembali tidak boleh sebelum tanggal pinjam. Bukti wajib pada create, tetapi nullable pada update. Tipe yang diizinkan hanya jpg, jpeg, png, dan pdf, dengan ukuran maksimal 2 MB. Pesan errornya juga saya tulis per aturan.

Di React, saya memakai helper http() untuk menyisipkan header X-XSRF-TOKEN, Accept: application/json, dan X-Requested-With: XMLHttpRequest. Helper ini juga mengubah respons 422 menjadi objek errors dan melempar HttpError untuk status lain. Dengan begitu, komponen tidak perlu menulis ulang logika CSRF dan parsing error di setiap request.



Gambar 18. Helper http() untuk CSRF, JSON, dan penanganan error respons.

State halaman menyimpan records, keyword, dialog, form, errors, dan message. loadData( ) memanggil GET /peminjaman/data setiap kali halaman dibuka atau setelah operasi berhasil. Pencarian di tabel dilakukan di sisi klien dengan useMemo karena jumlah data masih kecil; server tetap mengirim seluruh data lewat endpoint JSON.



Gambar 19. State, loadData(), dan pencarian sisi klien di halaman Peminjaman.

Saat form disubmit, saya menyusun FormData, menambahkan field satu per satu, lalu mengirim fetch() dengan method POST. Untuk update, saya tetap mengirim POST dengan field _method bernilai PUT karena FormData lebih aman diproses sebagai multipart; Laravel membaca _method sebagai method spoofing. Setelah respons sukses, dialog ditutup, data dimuat ulang, dan pesan sukses ditampilkan.



Gambar 20. Pengiriman form Peminjaman dengan FormData dan method spoofing.

Tabel menampilkan nama peminjam, judul buku, tanggal pinjam, tanggal kembali, status, dan tautan bukti. Tautan dibuka di tab baru dan ikonnya berubah antara gambar dan dokumen sesuai ekstensi. Dialog tambah dan edit memakai komponen yang sama; saat edit, data diambil lebih dulu dari GET /peminjaman/{id}/edit, lalu file dikosongkan supaya admin bisa memilih bukti baru.



Gambar 21. Struktur tabel peminjamans, termasuk kolom bukti dan nama_bukti.

3.7 Autentikasi dan Dashboard

Autentikasi tidak saya pindahkan ke dalam modul. Register, login, logout, verifikasi email, dan reset password tetap memakai route serta controller bawaan starter kit yang ditangani Fortify. Saya hanya memastikan akun baru mendapat role user dari nilai default kolom role dan atribut model User. Dengan begitu, modul tidak perlu tahu bagaimana proses login berjalan.

Dashboard berada di app/Http/Controllers/DashboardController.php dan memakai middleware auth serta verified. Isinya adalah jumlah buku, jumlah peminjaman, jumlah yang masih Dipinjam, dan jumlah user. Jumlah user hanya dikirim kalau yang membuka dashboard adalah admin; user biasa menerima null dan kartu itu tidak ditampilkan. Layout dan sidebar juga membaca shared props auth, jadi menu Kelola User hanya muncul untuk admin.

Halaman register dan login saya uji dengan input yang salah lebih dulu. Pesan validasi muncul di bawah input, dan setelah data benar barulah user masuk ke dashboard. Untuk membedakan

hak akses, saya memakai dua akun: admin@example.com dan user@example.com. Keduanya memakai password yang sama, tetapi menu dan tombol aksi yang terlihat berbeda.

3.8 Seeder dan Data Awal

Data awal dibuat lewat DatabaseSeeder. Seeder membuat satu akun admin dan satu akun user, lalu memanggil BukuDatabaseSeeder dan PeminjamanDatabaseSeeder. Saya memakai factory admin() untuk akun admin supaya role-nya jelas, sedangkan akun user memakai state default. Data buku dan peminjaman diisi terpisah agar jumlahnya mudah diatur saat demonstrasi.

Setelah seeder dijalankan, dashboard menampilkan 6 buku, 4 peminjaman, dan 2 user. Angka itu yang saya pakai untuk memeriksa apakah query dashboard benar. Saya juga memastikan akun user tidak bisa membuka /admin/users; kalau dibuka langsung, halaman 403 muncul, bukan halaman kosong atau error 500.

3.9 Pengujian dan Pemeriksaan Route

Pengujian memakai Pest. Saya menulis tes untuk enum dan middleware role, CRUD Buku, CRUD UserManagement termasuk pengaman akun sendiri, CRUD Peminjaman termasuk unggah berkas, autentikasi, dashboard, dan seeder. Tes memakai SQLite di memori dan Storage::fake, jadi tidak mengubah database pertemuan9. Dengan begitu, tes bisa dijalankan berulang tanpa merusak data demonstrasi.

Selain tes, saya memeriksa route dengan php artisan route:list --except-vendor. Saya memfilter per modul supaya daftar route tidak bercampur dengan route bawaan Fortify dan starter kit. Pemeriksaan ini membantu saya memastikan nama route, method, dan middleware sudah sesuai sebelum mengecek tampilan.

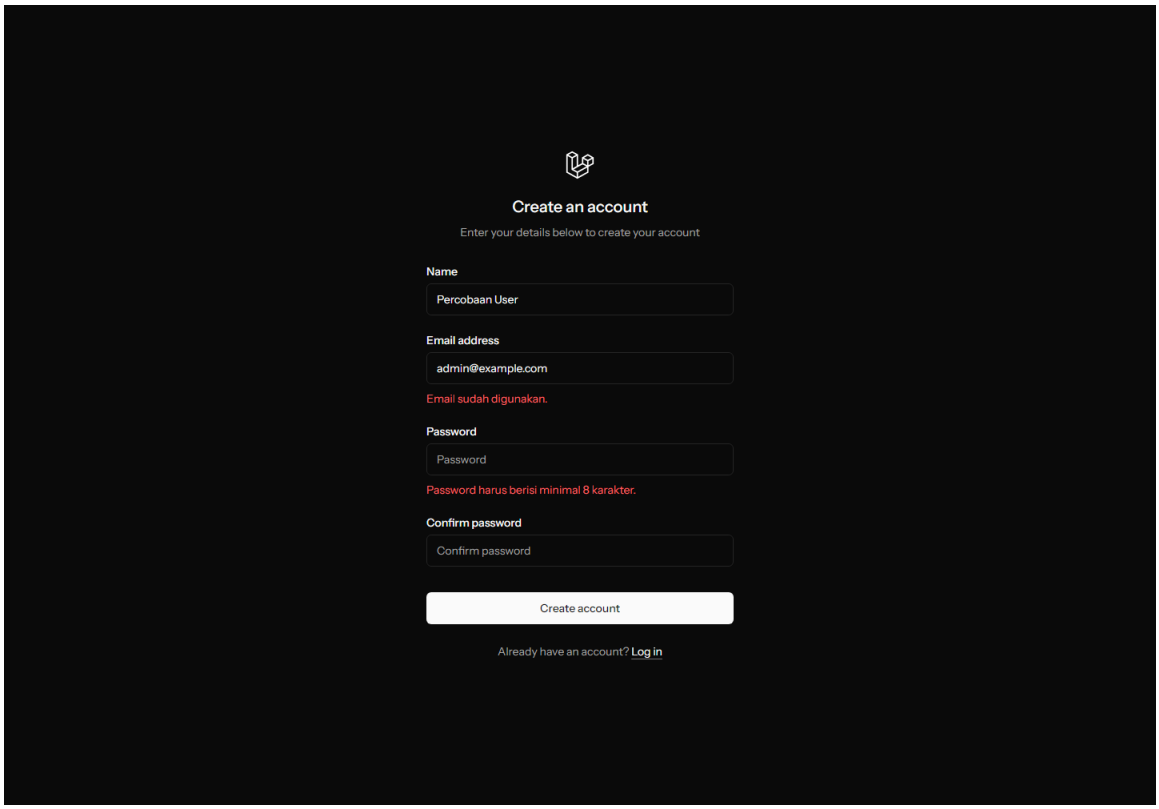
Hasil akhirnya 83 tes lulus dengan 408 assertion. Saya juga menjalankan npm run build, npm run lint, dan npx tsc --noEmit untuk memastikan halaman React tidak meninggalkan error TypeScript atau ESLint. Ketiga perintah itu selesai tanpa error.

4. Hasil dan Dokumentasi Program

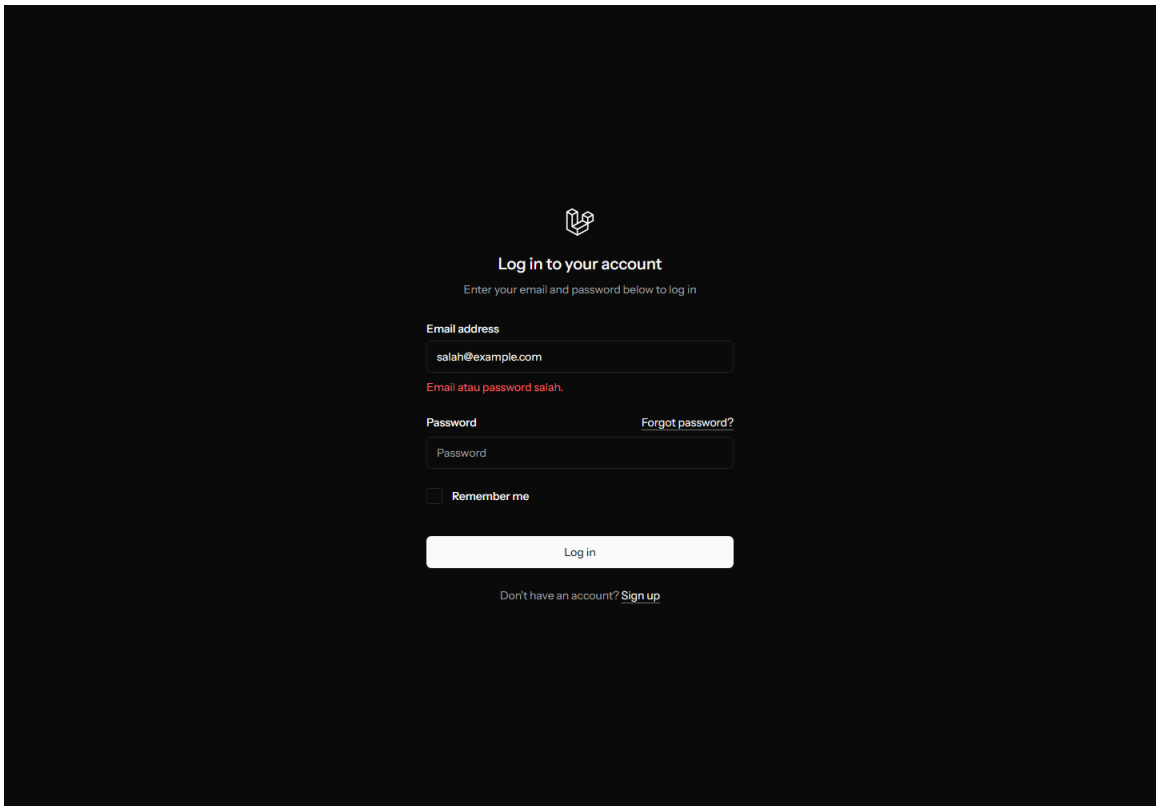
Bagian ini berisi tangkapan layar aplikasi yang berjalan di http://127.0.0.1:8000. Saya mengambil gambar dari Chrome headless dengan ukuran 1440 x 1000 piksel supaya tampilan desktop terlihat utuh. Setiap gambar saya beri keterangan singkat agar hubungannya dengan langkah di bagian sebelumnya jelas.

4.1 Autentikasi dan Dashboard

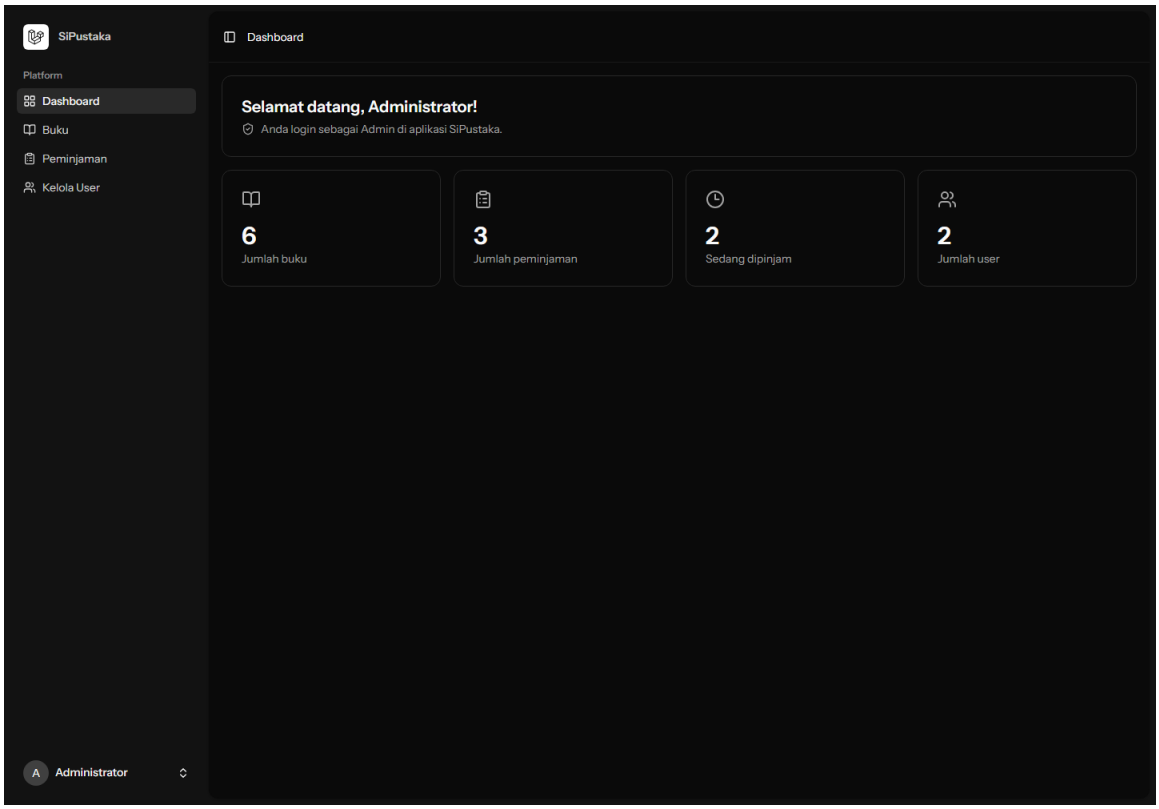
Halaman register dan login masih memakai tampilan starter kit. Saya sengaja tidak mengubah desainnya karena fokus modul ada di HMVC, role, dan AJAX. Yang saya ubah hanya data yang dikirim dan tujuan setelah login, yaitu dashboard.



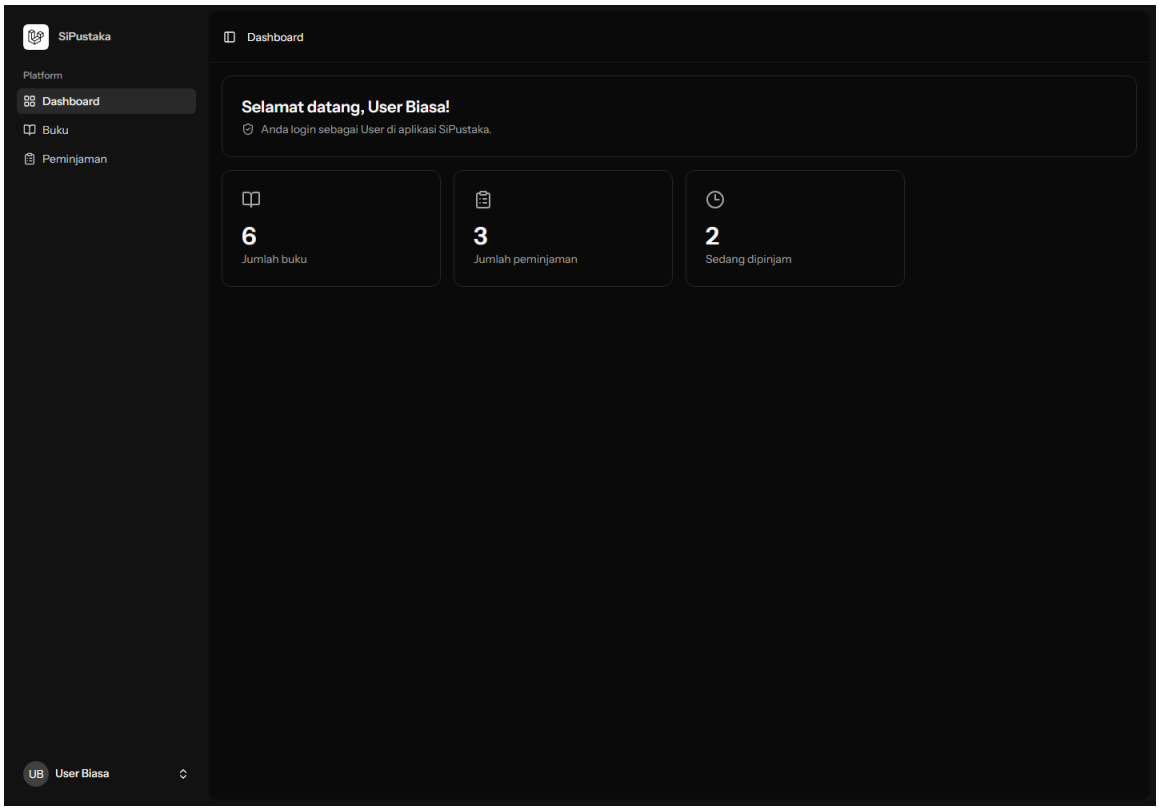
Gambar 22. Pesan validasi pada halaman register saat data belum lengkap.



Gambar 23. Pesan error pada halaman login saat email atau password salah.



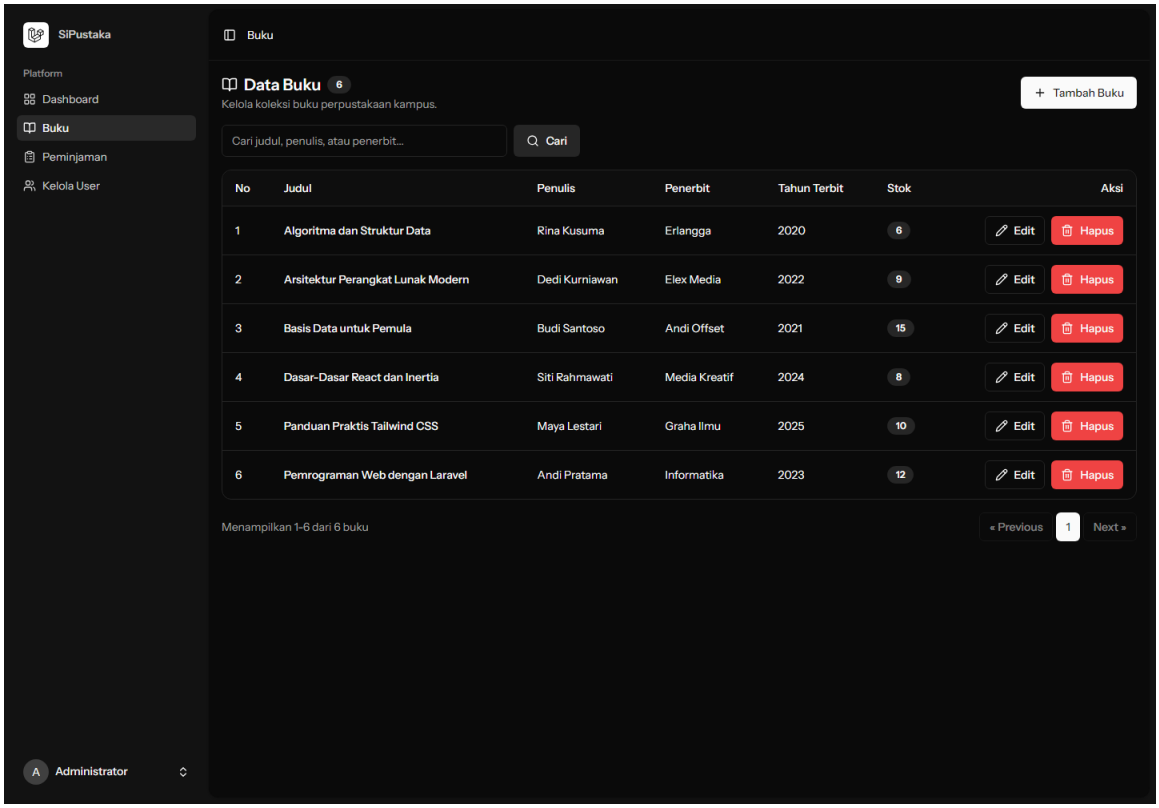
Gambar 24. Dashboard admin dengan ringkasan buku, peminjaman, dan jumlah user.



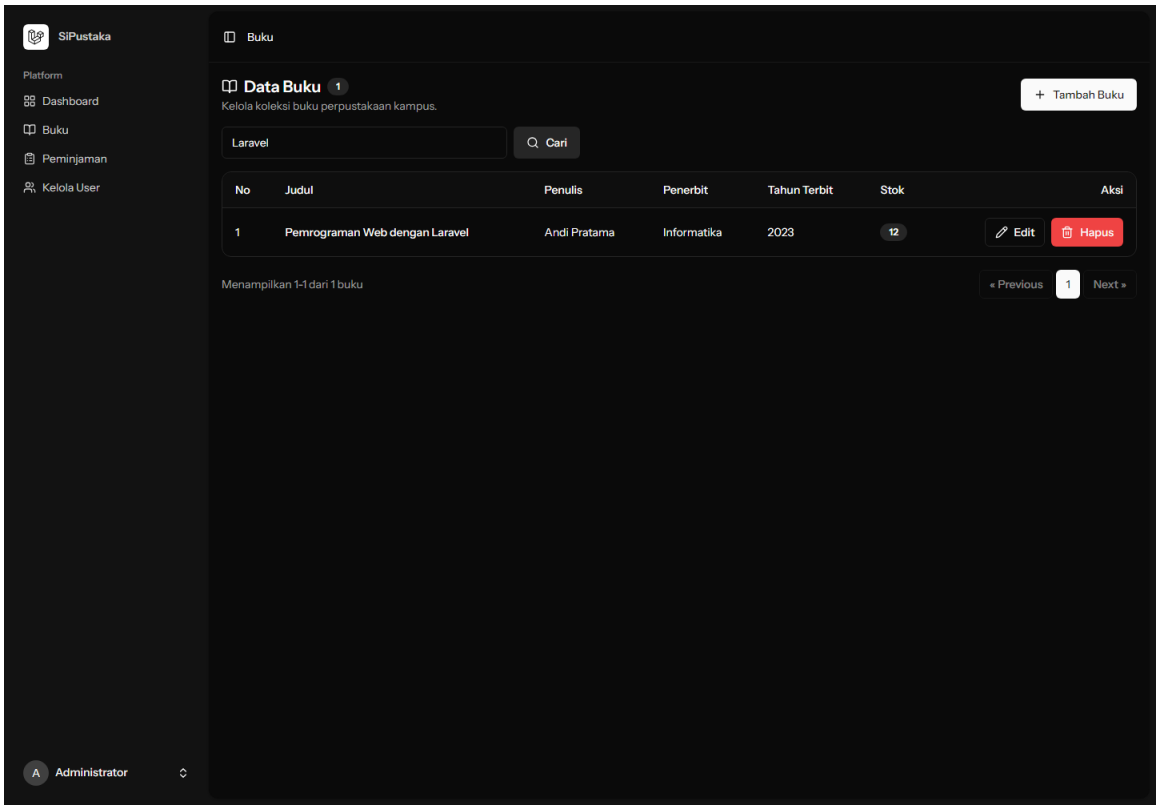
Gambar 25. Dashboard user biasa tanpa kartu jumlah user dan tanpa menu Kelola User.

4.2 Modul Buku

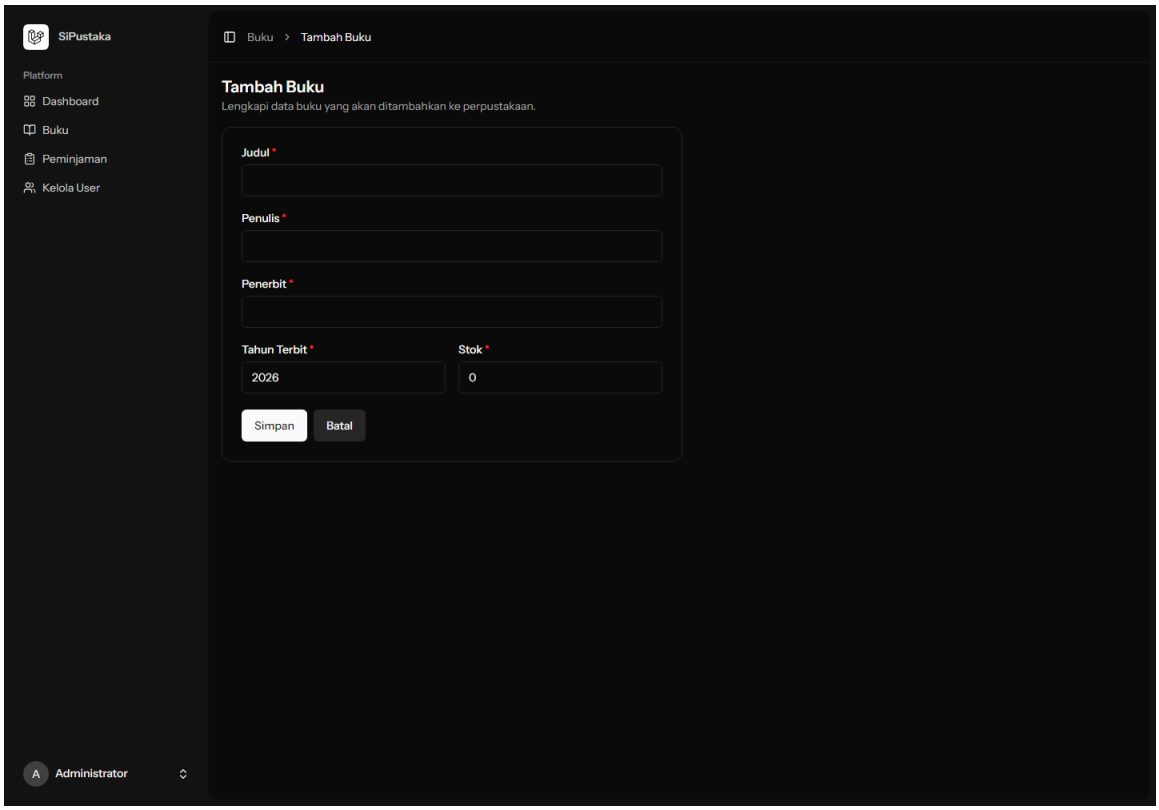
Modul Buku bisa dibuka oleh admin dan user. Perbedaannya ada pada tombol aksi: admin melihat Tambah, Edit, dan Hapus, sedangkan user hanya melihat tabel dan pencarian. Saya menampilkan keduanya supaya pembatasan role terlihat dari tampilan, bukan hanya dari route.



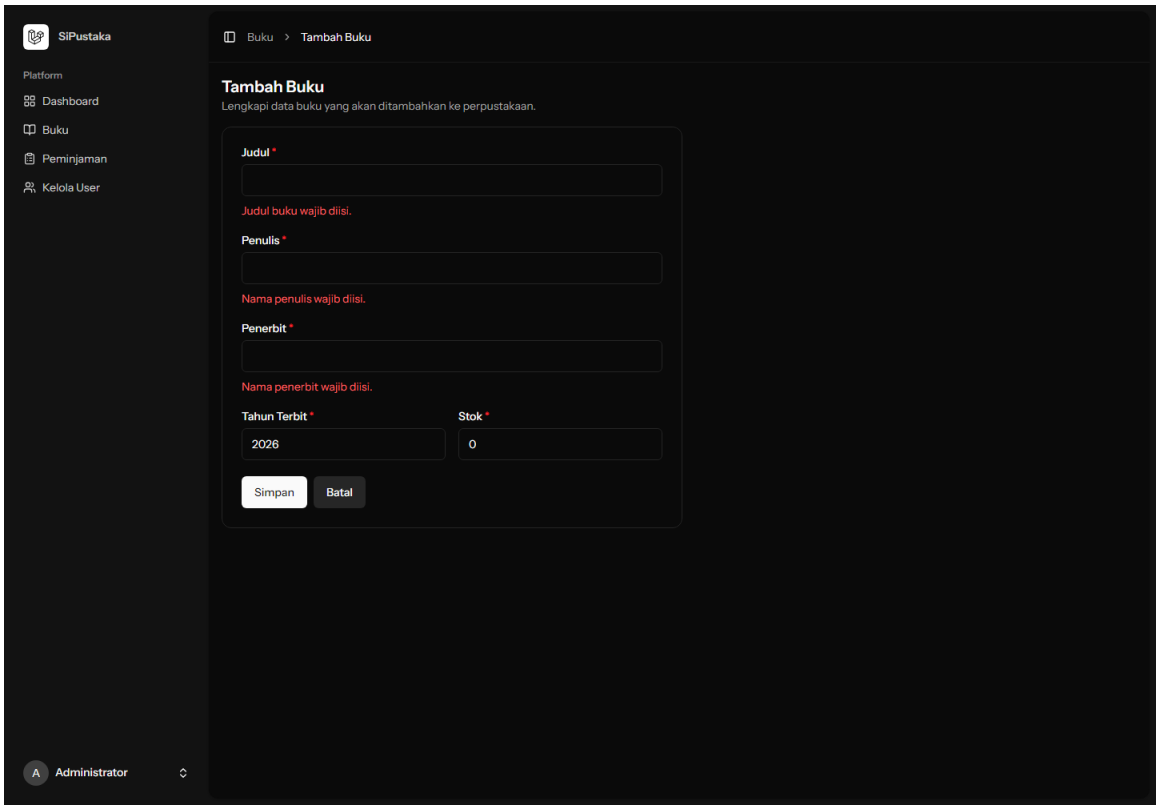
Gambar 26. Daftar buku yang terlihat oleh admin, lengkap dengan tombol aksi.



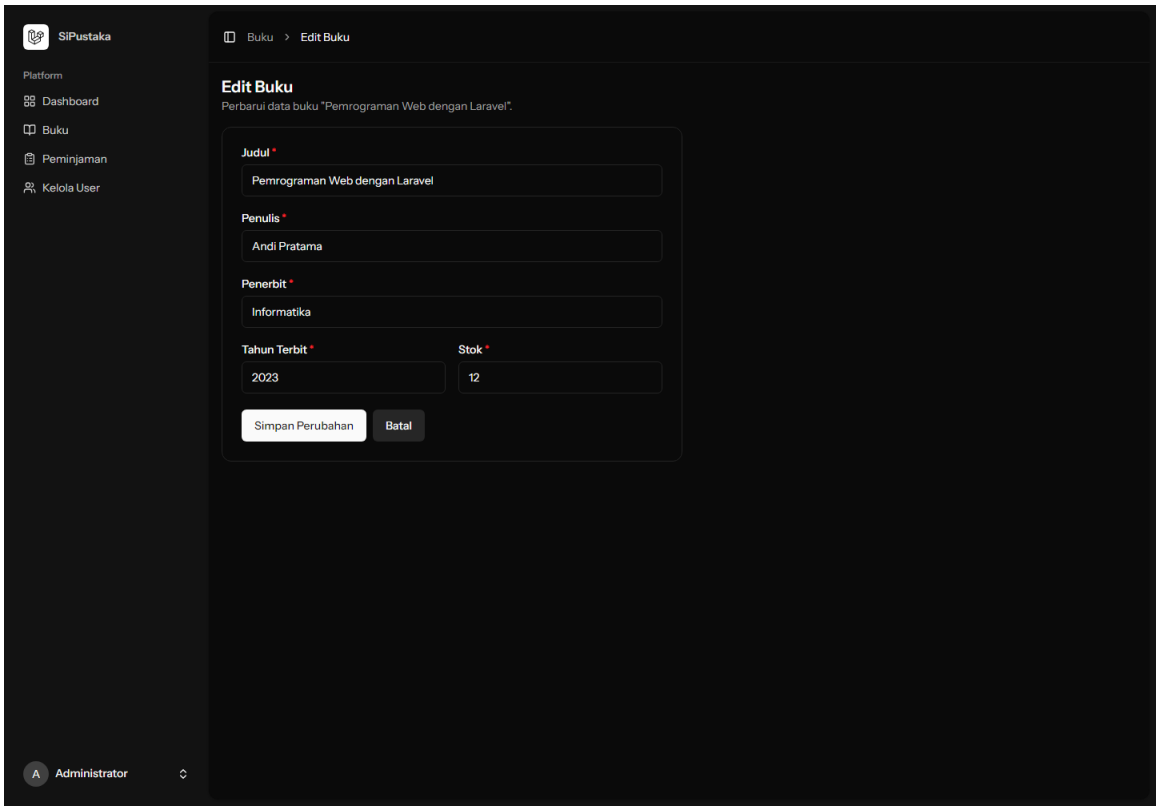
Gambar 27. Hasil pencarian buku dengan kata kunci judul.



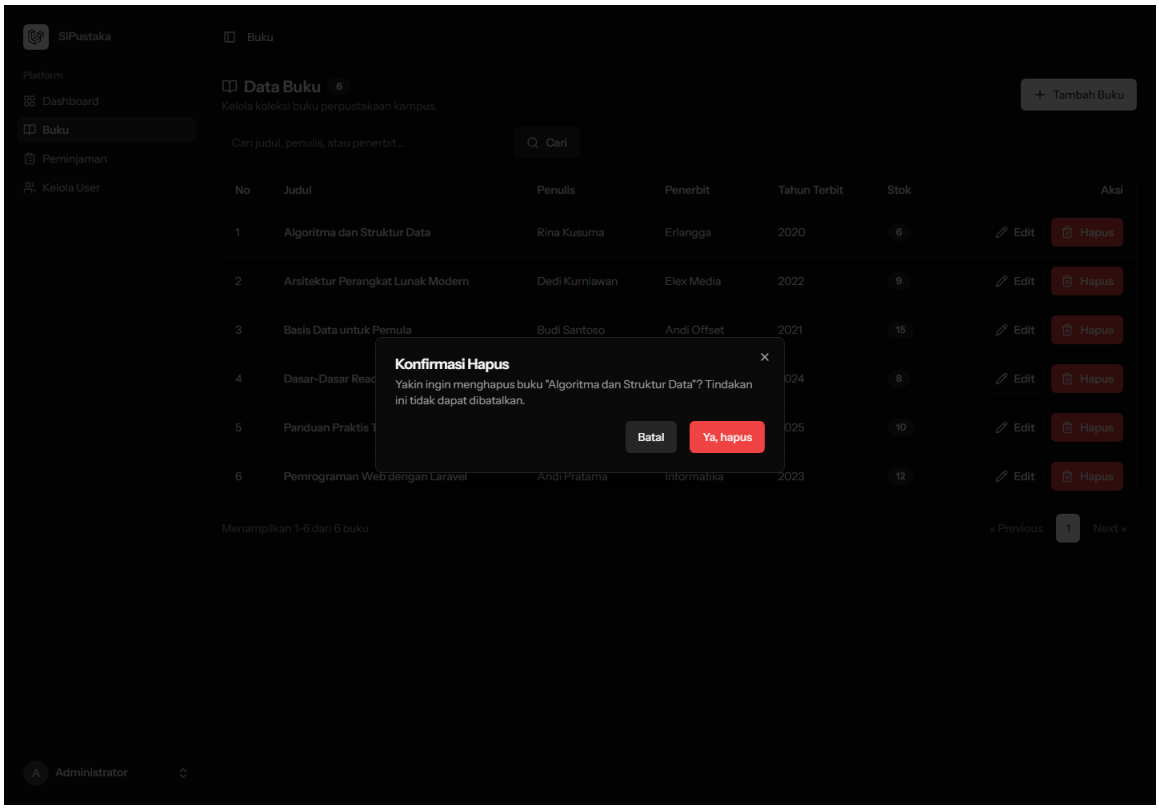
Gambar 28. Form tambah buku sebelum diisi.



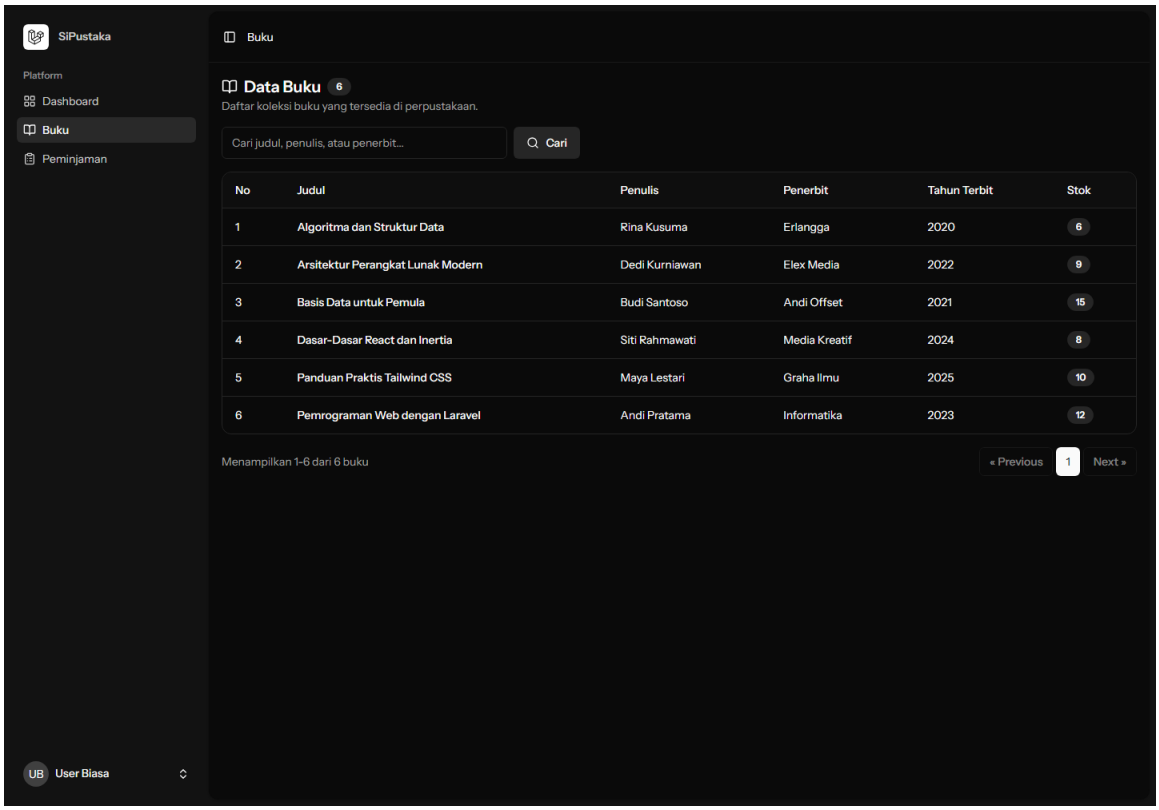
Gambar 29. Pesan validasi pada form tambah buku.



Gambar 30. Form edit buku dengan data yang sudah terisi.



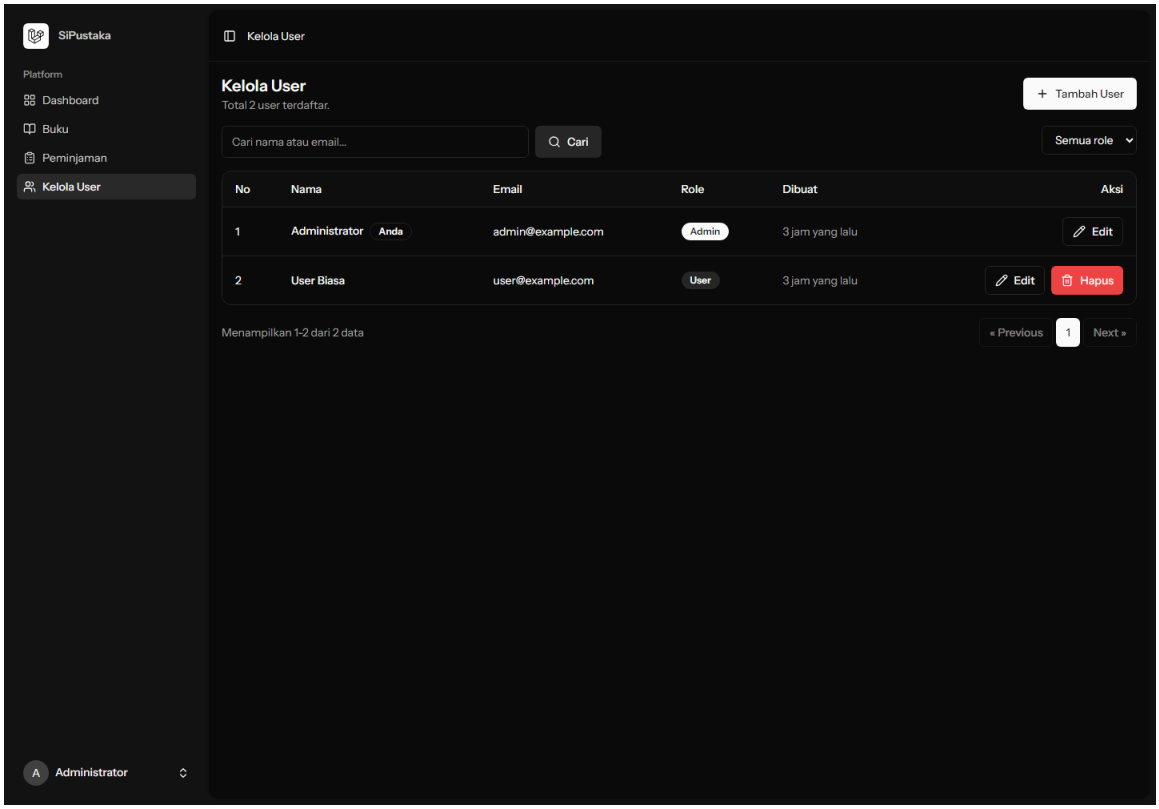
Gambar 31. Dialog konfirmasi sebelum buku dihapus.



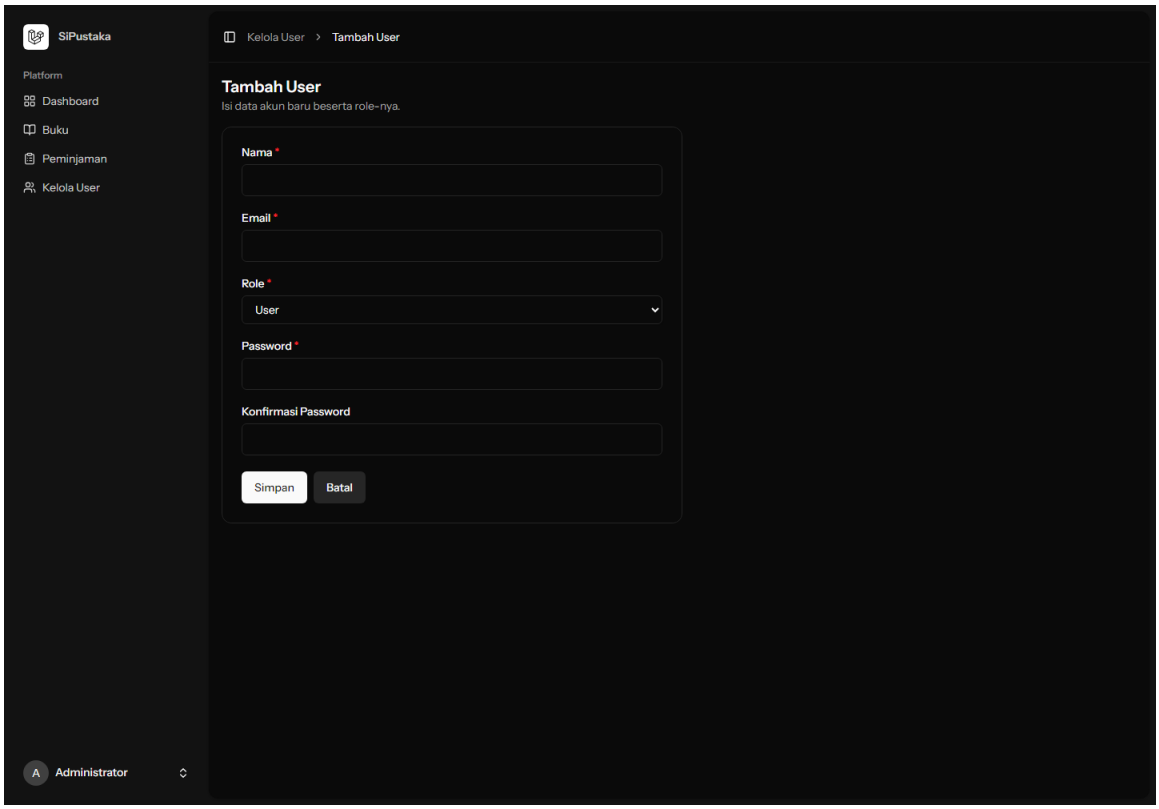
Gambar 32. Daftar buku yang terlihat oleh user biasa, tanpa tombol tambah, edit, atau hapus.

4.3 Modul UserManagement dan Otorisasi

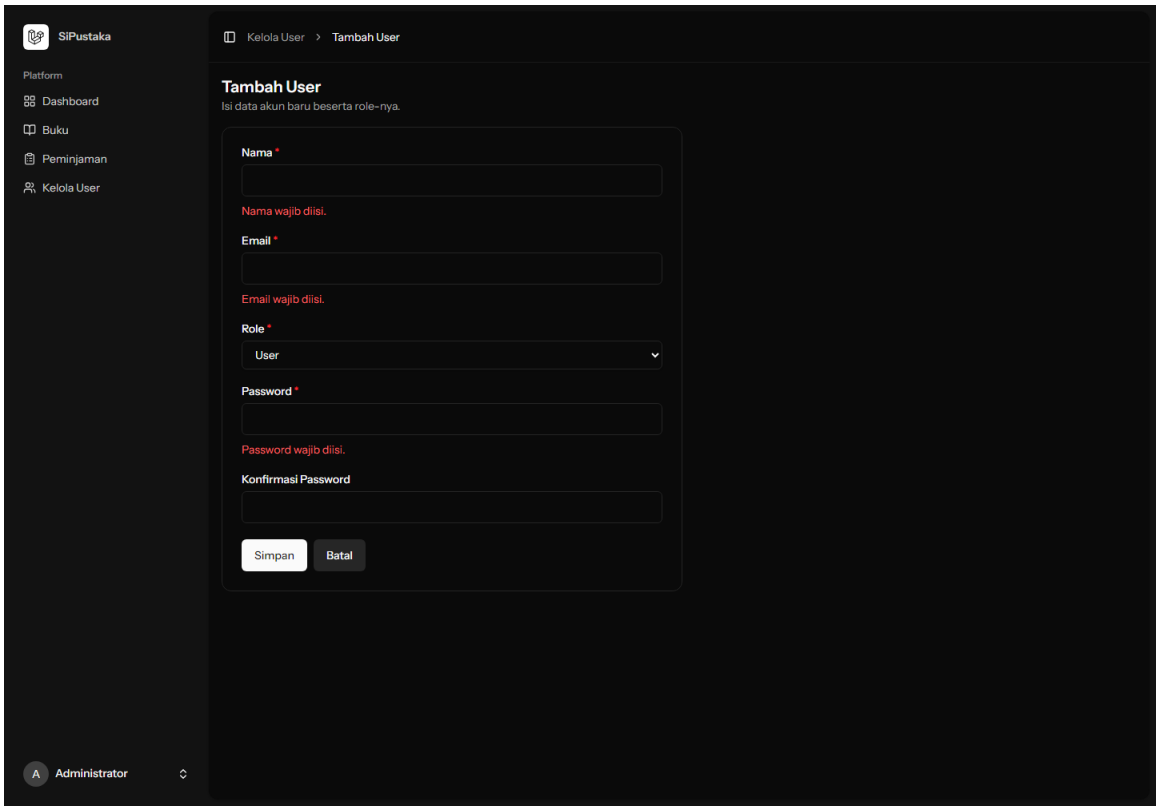
Halaman Kelola User hanya bisa dibuka admin. Saya menguji tiga hal di sini: pencarian dan filter role, validasi form, serta pengaman agar admin tidak menurunkan role atau menghapus akunnya sendiri. Halaman 403 saya sertakan sebagai bukti bahwa pembatasan tidak hanya ada di tampilan.



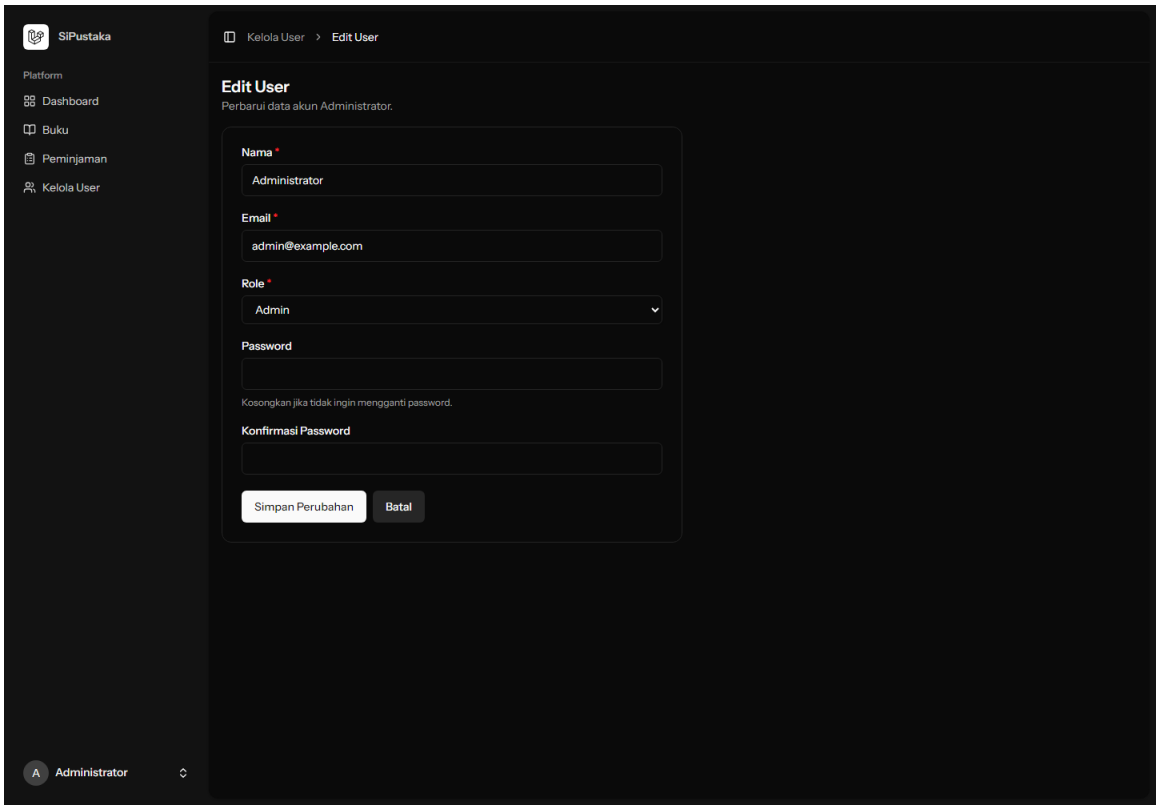
Gambar 33. Daftar user dengan pencarian, filter role, dan badge role.



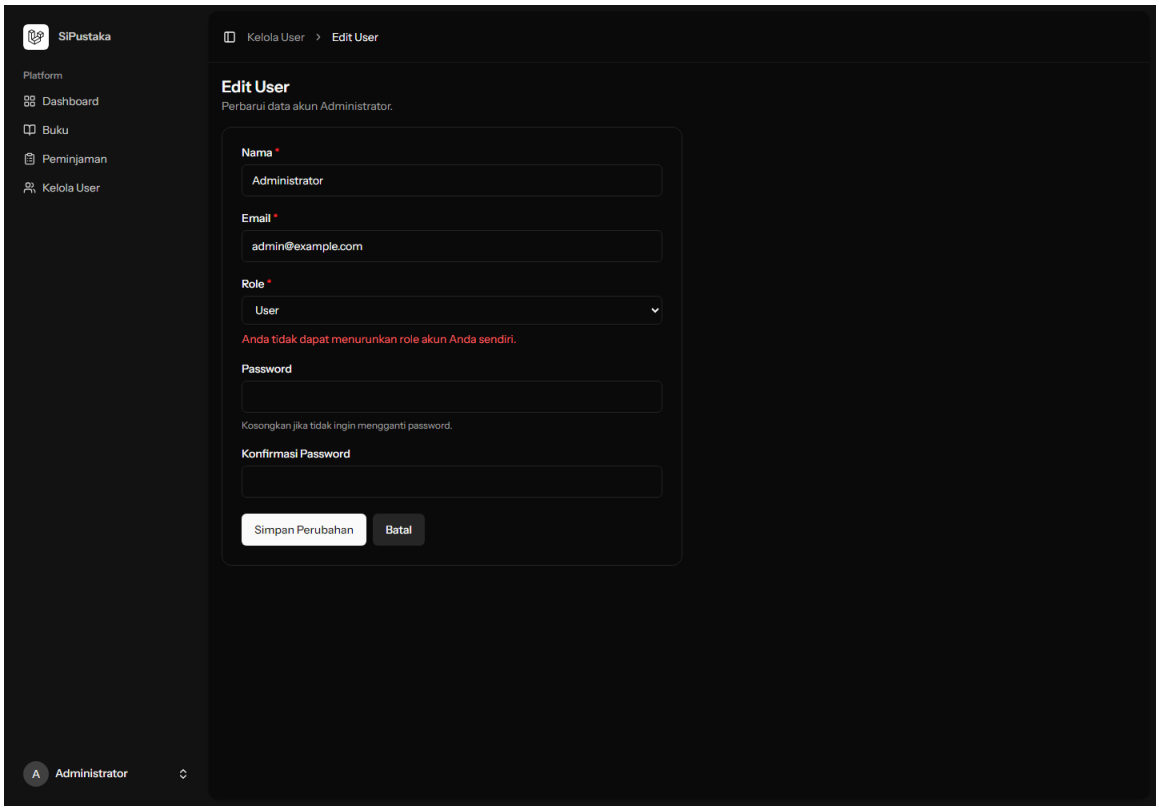
Gambar 34. Form tambah user dengan pilihan role.



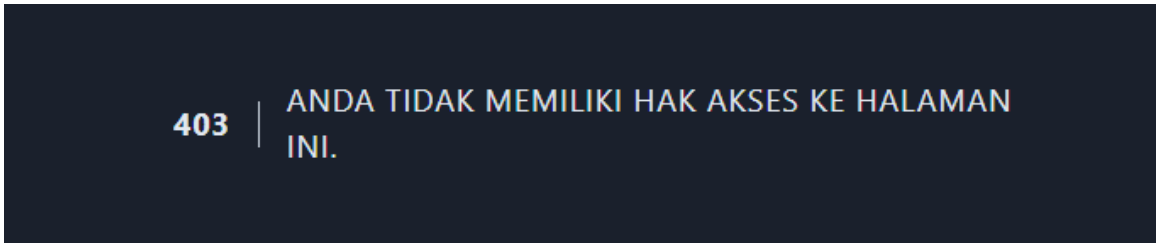
Gambar 35. Pesan validasi pada form tambah user.



Gambar 36. Form edit user; kolom password boleh dikosongkan.



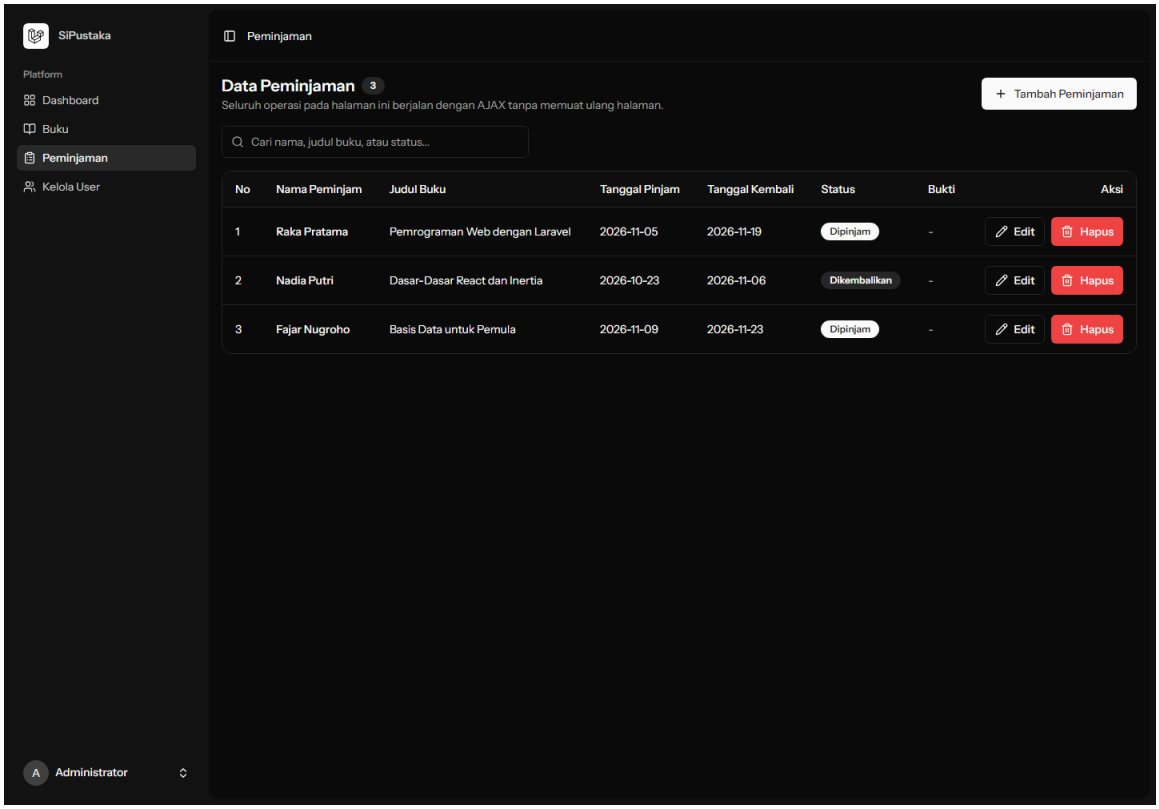
Gambar 37. Pesan error saat admin mencoba menurunkan role akunnya sendiri.



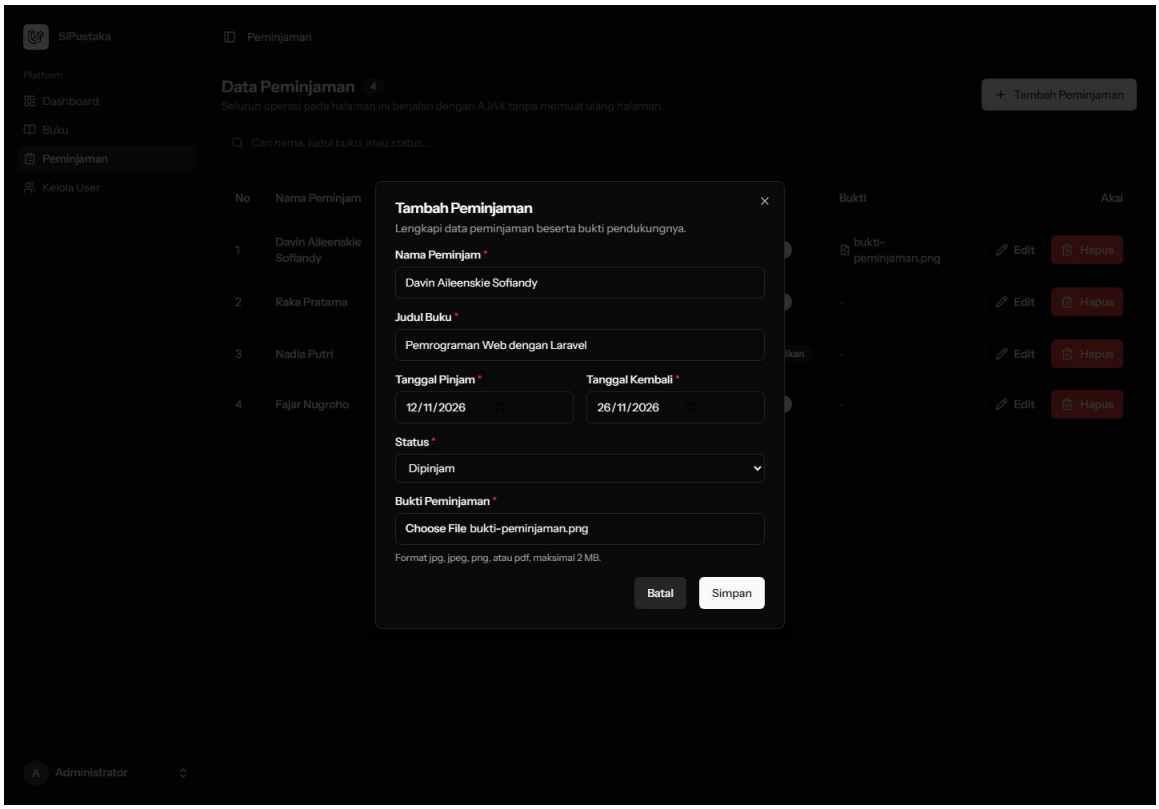
Gambar 38. Halaman 403 saat user biasa membuka /admin/users.

4.4 Modul Peminjaman Berbasis AJAX

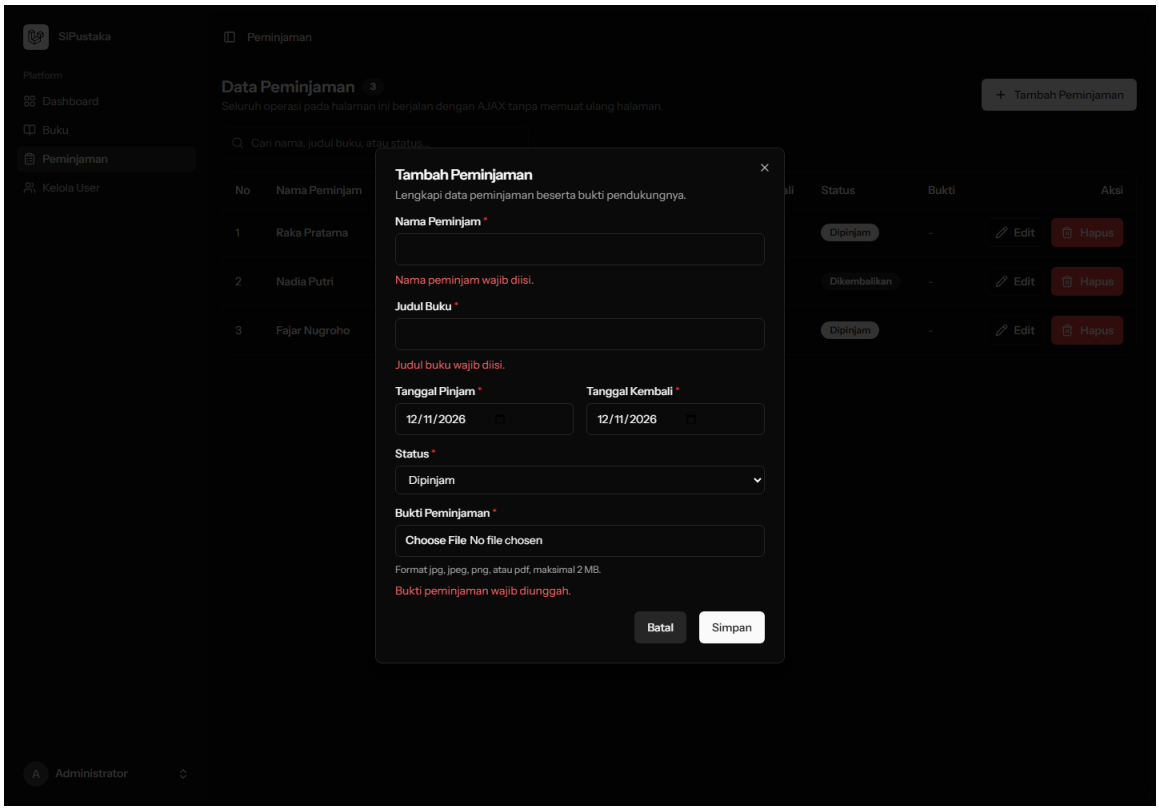
Peminjaman adalah bagian yang paling banyak saya uji karena semua operasi berjalan lewat fetch(). Tabel awal dimuat dari GET /peminjaman/data, form tambah dan edit memakai dialog, dan setiap respons sukses langsung memperbarui tabel tanpa reload. Saya juga menyertakan bukti validasi kosong, validasi tipe berkas, dan tab Network di DevTools.



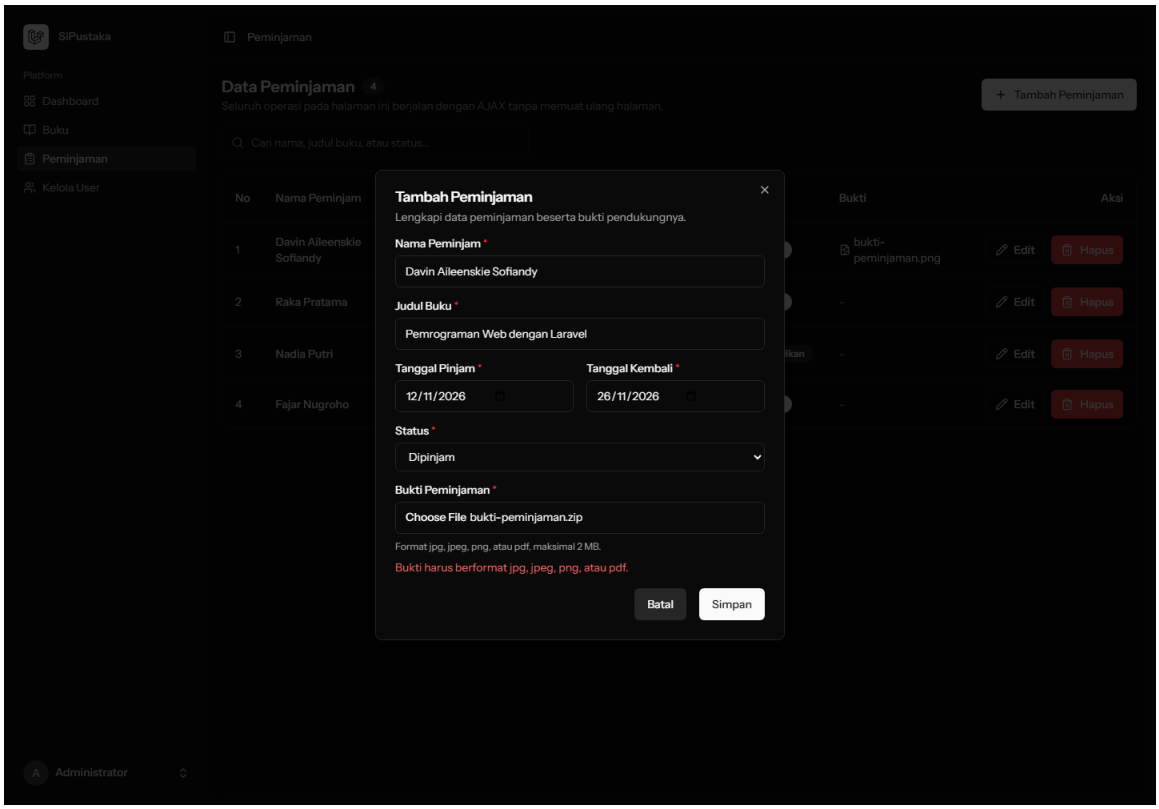
Gambar 39. Halaman Peminjaman dengan tabel data dan tombol tambah.



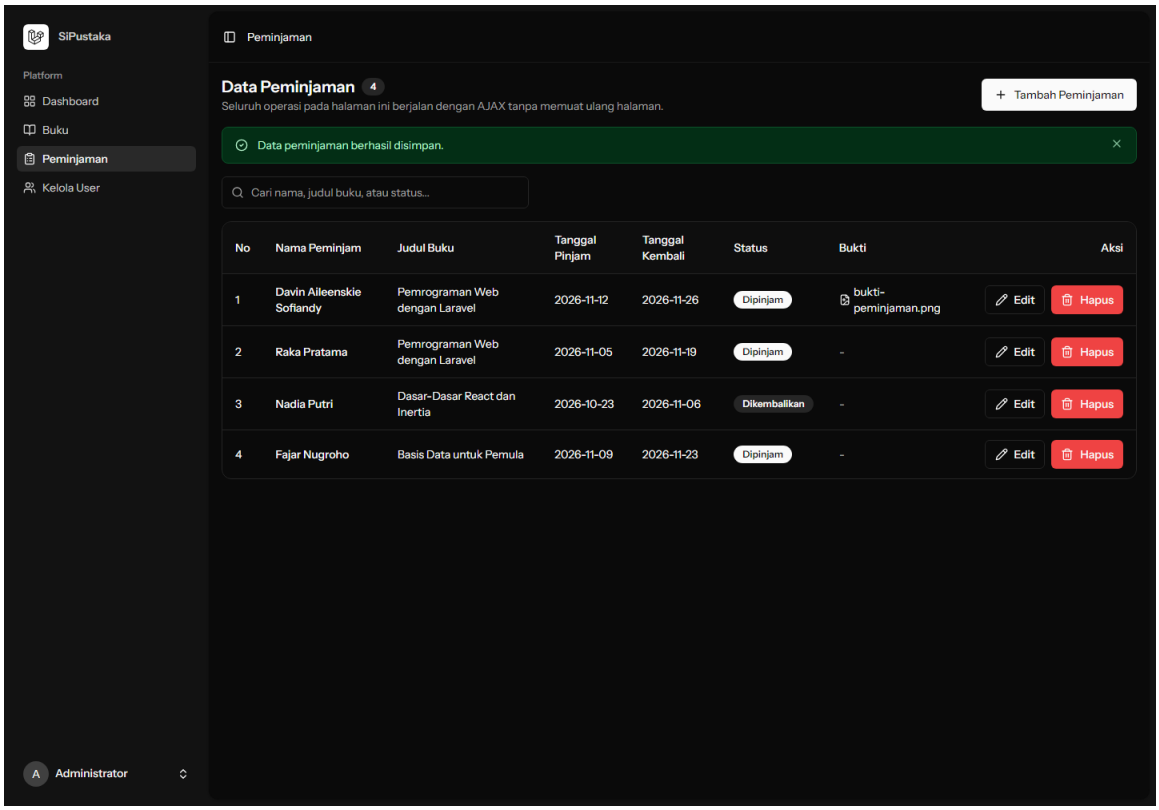
Gambar 40. Dialog tambah peminjaman yang sudah diisi dan berkas bukti dipilih.



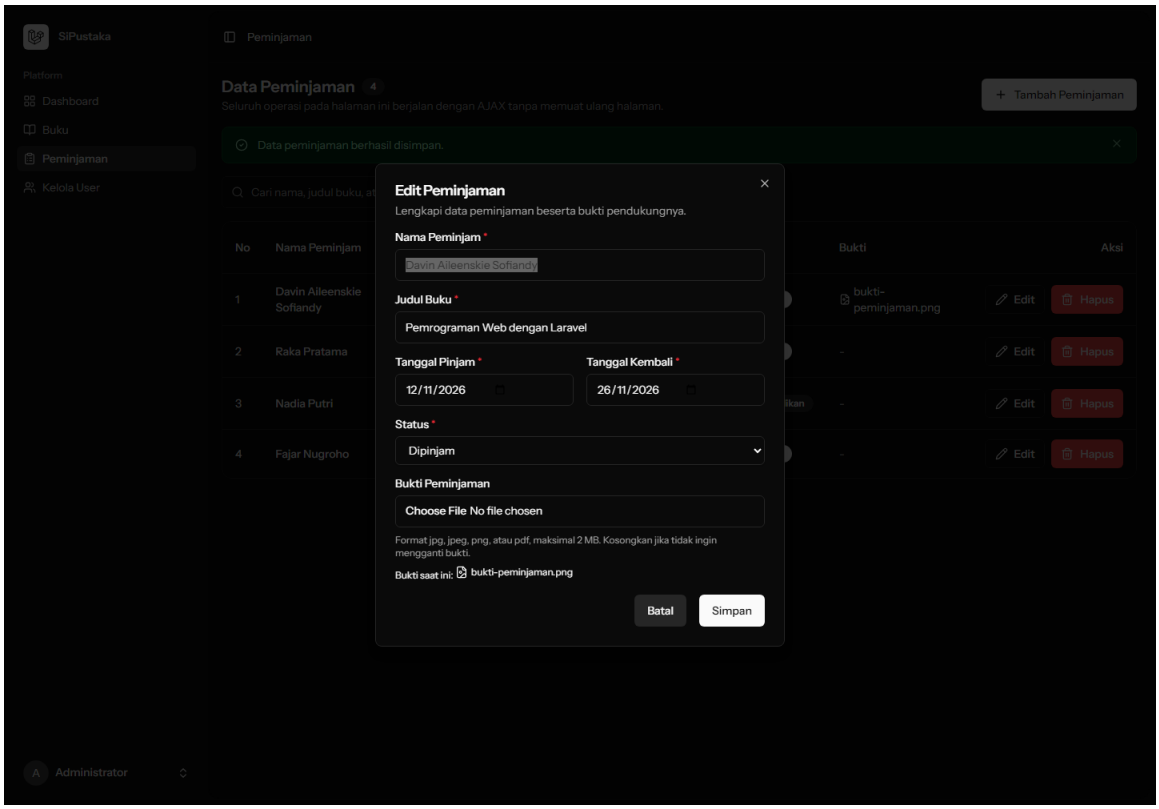
Gambar 41. Pesan validasi dari respons 422 JSON saat field wajib dikosongkan.



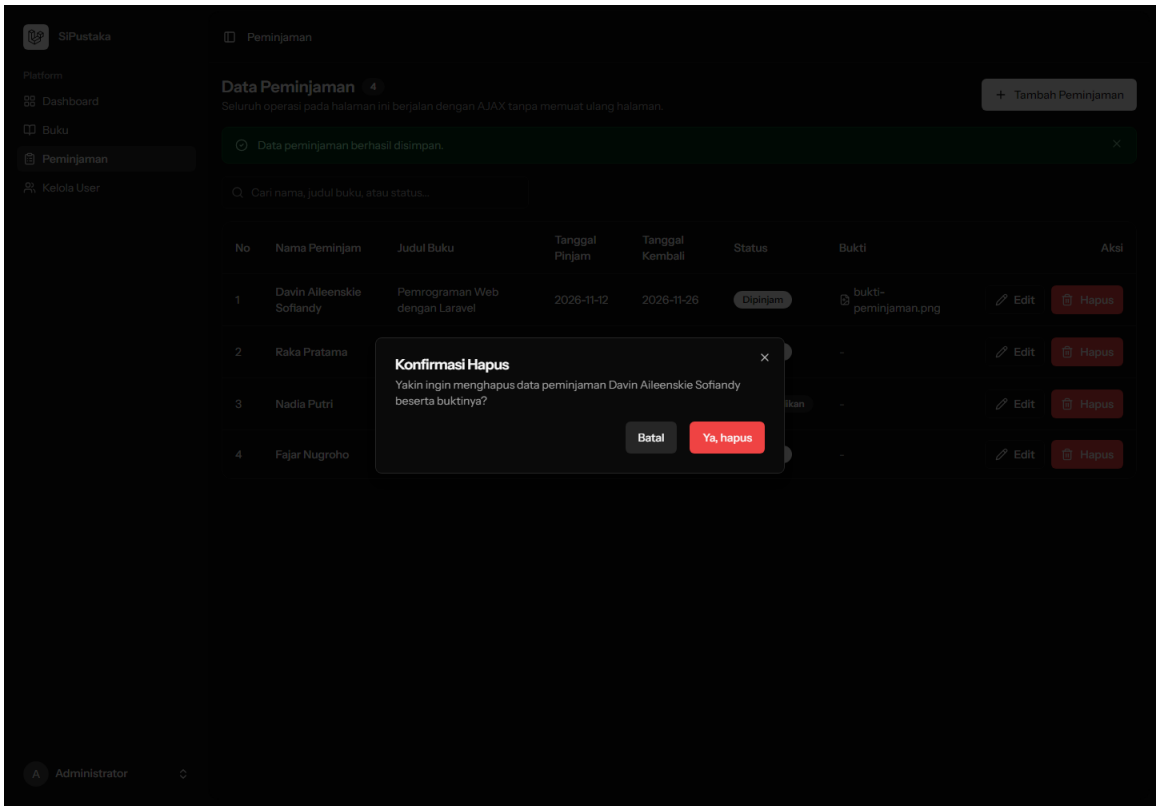
Gambar 42. Pesan validasi saat berkas ZIP diunggah; server hanya menerima jpg, jpeg, png, dan pdf.



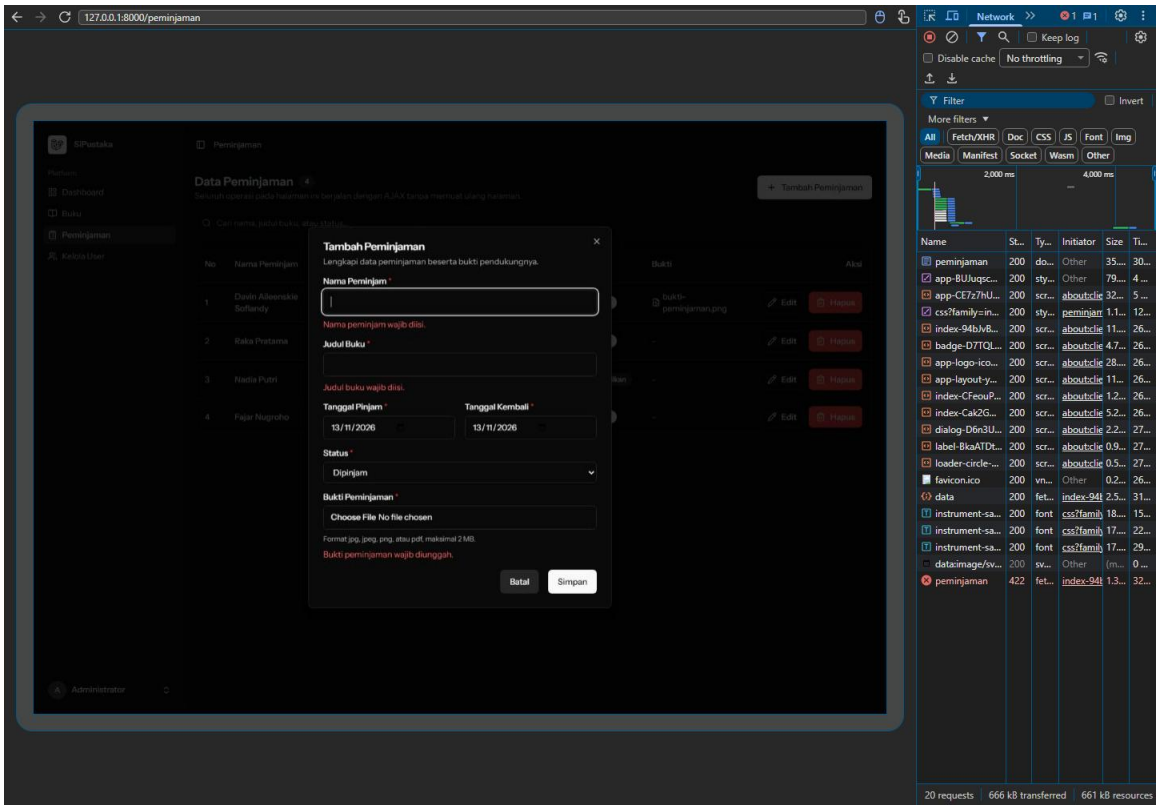
Gambar 43. Pesan sukses setelah data peminjaman berhasil disimpan.



Gambar 44. Dialog edit peminjaman dengan data lama dan pilihan bukti baru.



Gambar 45. Dialog konfirmasi sebelum data peminjaman dihapus.



Gambar 46. Tab Network di DevTools saat halaman memuat data dan saat request POST divalidasi.

4.5 Berkas dan Database

Berkas bukti disimpan di storage/app/public/bukti_peminjaman, lalu diakses lewat symlink public/storage. Saya memeriksa nama file hasil store dan mencocokkannya dengan kolom bukti di phpMyAdmin. Nama asli berkas tetap tersimpan di kolom nama_bukti, sedangkan path lengkapnya ada di kolom bukti.



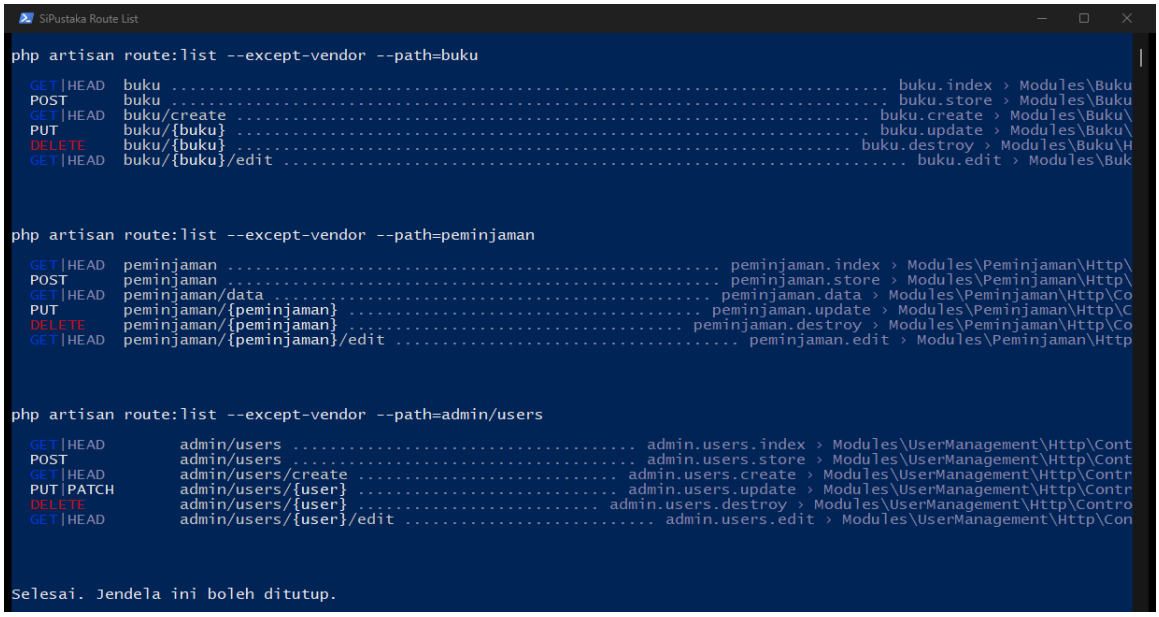
Gambar 47. Berkas bukti dengan nama acak di storage/app/public/bukti_peminjaman.

	status	bukti	nama_bukti	created_at	updated_at
	Dipinjam	*NULL*	*NULL*	2026-11-12 12:36:33	2026-11-12 12:36:33
	Dikembalikan	*NULL*	*NULL*	2026-11-12 12:36:33	2026-11-12 12:36:33
	Dipinjam	*NULL*	*NULL*	2026-11-12 12:36:33	2026-11-12 12:36:33
	Dipinjam	bukti_peminjaman/sBXr4sVUX51WVxf7JkrPkOqRR0lGfDxHLhnSklOl.png	bukti-peminjaman.png	2026-11-12 15:40:54	2026-11-12 15:40:54

Gambar 48. Kolom bukti dan nama_bukti pada tabel peminjamans di phpMyAdmin.

4.6 Route, Modul, dan Pengujian

Saya memeriksa route per modul dengan php artisan route:list --except-vendor. Daftar itu menunjukkan method, URI, nama route, controller, dan middleware yang terpasang. Untuk status modul, hasil php artisan module:list sudah saya tampilkan di bagian 3.1. Pengujian otomatis dijalankan dengan php artisan test; hasilnya seluruh tes lulus.



Gambar 49. Sebagian hasil php artisan route:list --except-vendor untuk route Buku, Peminjaman, dan admin/users.



Gambar 50. Hasil php artisan test: 83 tes lulus dengan 408 assertion.

5. Analisis dan Pembahasan

1. Pemetaan HMVC pada SiPustaka.

HMVC di sini bukan berarti setiap modul berjalan sebagai aplikasi terpisah. Semua modul masih memakai satu database, satu konfigurasi, dan satu proses Laravel. Yang dipisahkan adalah kode: controller, request, model, route, migration, dan halaman React milik Buku, Peminjaman, dan UserManagement berada di dalam folder Modules. Autentikasi, dashboard, layout, dan komponen UI tetap berada di app dan resources.

Pemisahan itu membuat batas tanggung jawab lebih jelas. Modul Buku tidak perlu tahu bagaimana peminjaman dicatat, dan modul Peminjaman tidak perlu tahu bagaimana akun dikelola. Ketiganya hanya bertemu di model User dan komponen tampilan bersama. Dashboard menjadi pengecualian kecil karena controller-nya berada di luar modul dan mengimpor model Buku serta Peminjaman untuk menghitung ringkasan. Saya membiarkannya karena dashboard memang milik aplikasi, bukan milik salah satu modul.

2. Autentikasi di luar modul, otorisasi di dalam route.

Autentikasi tetap memakai Fortify dan route bawaan starter kit. Saya tidak menyalin logika login ke dalam modul karena itu akan menduplikasi kode yang sudah teruji. Yang saya tambahkan adalah role dan middleware. Dengan pola ini, modul cukup memakai middleware auth dan role, lalu menyerahkan proses login dan logout ke starter kit.

Otorisasi saya taruh di route, bukan hanya di tombol React. Tombol yang disembunyikan memang membuat tampilan lebih bersih, tetapi tidak bisa dijadikan pengaman. Karena itu route Buku dan UserManagement memakai role:admin untuk operasi tulis, sedangkan Peminjaman memakai role:admin,user karena kedua role boleh mencatat peminjaman.

3. Pembatasan akses berlapis.

Ada tiga lapis pembatasan yang saya pakai. Lapis pertama adalah middleware auth, yang mengarahkan user yang belum login ke halaman login. Lapis kedua adalah middleware role, yang mengembalikan 403 kalau role tidak cocok. Lapis ketiga adalah authorize() di request class dan pengecekan di controller untuk aturan yang lebih spesifik, seperti admin tidak boleh menurunkan role sendiri.

Lapis ketiga penting karena aturan seperti itu tidak bisa ditangani middleware role biasa. Middleware hanya tahu role user, bukan apakah user sedang mengedit akunnya sendiri. Dengan menaruh aturan di UpdateUserRequest dan destroy(), saya bisa memakai pesan validasi yang jelas dan tetap aman kalau request dikirim langsung.

4. CRUD Inertia dan CRUD AJAX punya peran berbeda.

Buku dan UserManagement memakai Inertia karena halaman-halaman itu memang berbentuk halaman penuh dengan form dan redirect. Inertia membuat saya tetap menulis controller dan request seperti Laravel biasa, tetapi tampilannya React. Flash session dan error validasi juga lebih mudah karena Inertia sudah membawa props tersebut.

Peminjaman memakai AJAX karena saya ingin tabel yang berubah tanpa reload dan dialog yang bisa dibuka tanpa pindah halaman. Konsekuensinya, semua respons harus JSON, error 422 harus diparsing sendiri, dan CSRF harus dikirim lewat header. Saya menaruh logika itu di helper http() supaya komponen tidak penuh dengan kode fetch berulang.

5. Unggah dan pengelolaan bukti.

Bukti peminjaman disimpan di disk public, bukan di disk lokal, karena harus bisa diakses lewat URL. Nama file hasil store berupa hash, jadi saya menyimpan nama asli di kolom nama_bukti. Saat berkas diganti, file lama dihapus lebih dulu; saat data dihapus, file juga ikut dihapus. Ini mencegah folder bukti_peminjaman menumpuk file yang tidak lagi dipakai.

Validasi berkas dilakukan di server, bukan hanya di atribut accept pada input. Saya membuktikannya dengan mengunggah file ZIP: browser tetap bisa mengirim file itu lewat FormData, tetapi server menolaknya dengan pesan yang jelas. Batas 2 MB juga saya uji, walaupun screenshot yang saya sertakan fokus pada kesalahan tipe berkas.

6. Validasi dan pesan error.

Pesan validasi saya tulis dalam bahasa Indonesia dan ditempel di bawah input yang bermasalah. Untuk Inertia, error datang sebagai props errors; untuk AJAX, error datang dari respons 422 JSON. Saya memakai nama field yang sama di kedua jalur supaya komponen InputError bisa dipakai ulang. Pesan sukses juga konsisten: flash session untuk Inertia dan field success pada JSON untuk Peminjaman.

7. Pengujian dan batasannya.

Pengujian otomatis memberi kepercayaan lebih besar daripada mencoba satu per satu di browser. Saya menguji role, middleware, CRUD, unggah berkas, dan seeder. Tes memakai SQLite di memori dan Storage::fake, jadi tidak menyentuh database pertemuan9. Hasil akhirnya 83 tes lulus dengan 408 assertion.

Batasannya, pengujian otomatis tidak memeriksa tampilan secara visual. Karena itu saya tetap mengambil screenshot untuk halaman penting dan memeriksa DevTools Network. Screenshot dan tes saling melengkapi: tes memastikan logika, sedangkan screenshot memastikan tampilan dan alur yang saya laporkan benar-benar ada.

8. Deviasi dari teks modul.

Ada beberapa hal yang berbeda dari teks modul. Pertama, nama project dan database. Modul menyebut sipustaka_hmvc_react dan db_sipustaka, sedangkan yang saya pakai adalah PRK_TA_9 dan pertemuan9. Saya tidak mengganti nama karena project dan data dari modul sebelumnya sudah berjalan di sana. Kedua, modul contoh di teks adalah UserManagement dan Pendaftaran, sedangkan aplikasi saya memakai Buku, Peminjaman, dan UserManagement. Ketiga, saya memakai React 19 dan Inertia 2 yang sudah terpasang di starter kit, bukan versi yang lebih lama.

Perbedaan itu tidak mengubah materi yang diuji. HMVC, role, middleware, CRUD Inertia, CRUD AJAX, unggah berkas, validasi, dan pengujian tetap saya terapkan. Saya mencatat deviasi ini supaya pembaca tidak mengira ada langkah yang terlewat.

9. Hal yang bisa diperbaiki.

Kalau waktu praktikum masih ada, saya ingin memindahkan dashboard ke modul tersendiri supaya tidak ada impor model lintas modul dari app. Saya juga ingin menambahkan policy untuk aturan yang lebih rinci, misalnya user hanya boleh mengubah peminjaman miliknya sendiri. Untuk Peminjaman, pencarian sisi klien masih cukup karena datanya sedikit; kalau data bertambah, pencarian dan pagination sebaiknya dipindahkan ke server seperti di Buku.

6. Kesimpulan

Struktur HMVC berhasil diterapkan pada Laravel React Starter Kit dengan paket nwidart/laravel-modules. Modul Buku, Peminjaman, dan UserManagement berdiri sebagai folder terpisah dengan route, controller, request, model, migration, dan halaman React sendiri. Autentikasi,

dashboard, dan layout tetap berada di luar modul. Halaman modul bisa dipanggil lewat format NamaModul::halaman berkat resolver di resources/js/app.tsx.

Otorisasi berbasis role berjalan di tiga lapis: middleware auth, middleware role, dan aturan khusus di request/controller. Admin bisa mengelola buku dan user, user biasa hanya bisa membaca buku dan mencatat peminjaman. Halaman 403 muncul saat user biasa membuka /admin/users, dan admin tidak bisa menurunkan role atau menghapus akunnya sendiri.

CRUD Inertia dan CRUD AJAX sama-sama berjalan. Buku dan UserManagement memakai Inertia dengan flash session dan validasi props, sedangkan Peminjaman memakai fetch( ), respons JSON, FormData, dan method spoofing. Bukti peminjaman tersimpan di storage/app/public/bukti_peminjaman, path-nya tercatat di kolom bukti, dan nama aslinya di kolom nama_bukti. Pengujian Pest mencatat 83 tes lulus dengan 408 assertion, dan aplikasi berjalan tanpa error di http://127.0.0.1:8000.

7. Link Repository Github

https://github.com/davinaileenskie1-hue/PABW-Semester-3